

Analysis and assessment of weighted combinatorial criterion for test suite reduction

Chi-Lun Chiang¹ | Chin-Yu Huang^{1,2}  | Chang-Yu Chiu² | Kai-Wen Chen² | Chen-Hua Lee¹

¹ Institute of Information Systems and Applications, National Tsing Hua University, Hsinchu, Taiwan

² Department of Computer Science, National Tsing Hua University, Hsinchu, Taiwan

Correspondence

Chin-Yu Huang, Institute of Information Systems and Applications, National Tsing Hua University, Hsinchu, Taiwan.
Email: cyhuang@cs.nthu.edu.tw

Funding information

Ministry of Science and Technology, Taiwan, Grant/Award Numbers: MOST 108-2221-E-007-033-MY3, MOST 110-2221-E-007-035-MY3

Abstract

Software plays an important role in modern society. New software features have to be completed and tested rapidly due to business and user needs and goals. To ensure the quality of modified software components, new test cases must be generated and included in the current test pool. Furthermore, test pool size often grows too large to test all cases. A considerably large test pool typically consumes an exorbitant quantity of time, leading to inefficient regression testing. Test suite reduction is often used to solve this problem by removing redundant test cases. Thus, time and resource constraints make it necessary to minimize the test suite. That is, test pool size should be efficiently reduced, while the remaining test cases should be adequate to provide the same coverage as the pool. The main purpose of this paper is to propose four weighted combinatorial algorithms and a three-objective binary integer linear programming (ILP) model to improve fault detection effectiveness. We first consider both the control flow and data flow as our testing criteria and combine them by using a weighted means with four existing test suite reduction algorithms. A genetic algorithm is used to find the best combination of weighting factors to assign to the two testing criteria. Experiments based on real subject programs are presented. Experimental results demonstrate that 77.5% of our proposed weighted combinatorial algorithms have an even or better performance in suite size reduction, 83.75% of our approaches perform even or better in fault detection effectiveness, and 82.5% of our approaches have even or better effectiveness in fault-to-test ratio. For the three-objective binary ILP model, test suite size is significantly reduced without any accompanying loss in fault detection ability.

KEYWORDS

fault detection effectiveness, genetic algorithm, regression testing, test suite minimization, test suite reduction, testing criteria, weighted combination

1 | INTRODUCTION

Software has become an important part of daily life, critical to human livelihoods and security. During the software development life cycle (SDLC), developers and project managers typically require a rigorous structured design with a verification process to ensure developed software product quality. Software testing¹ is commonly used to verify whether developed software meets users' requirements. In SDLC, static and dynamic testing are two different methods of detecting software bugs or defects. Static testing techniques test software products manually, without execution. On the other hand, dynamic testing techniques require computers for testing. Static and dynamic testing are actually complementary, with each type having its own strengths and unique approaches to detecting software faults. In general, an error represents human mistakes that can result in a fault.² Thus, a fault is caused by an error, and a failure occurs when a fault is executed; that is, human error will introduce faults into code, and program execution determines which code is being executed at any given time. Failure usually relates to the behavior of the program. In fact, there are many fault types in a program that cause failures, such as syntax, computation, documentation, overload, boundary, timing, synchronization issues, and so on.³ In practice, faults are hidden in the code and may never be seen in operation. If the code that contains faults is never executed in operation or uncovered during testing, the program never fails.

Dynamic testing is a more formal approach for test case design, coverage measurement, test execution, and test incident reporting. Dynamic testing may begin when the program is 100% completed. It observes the output and makes a judgment and passes a verdict. It has been noted that a test case contains predetermined input, anticipated output, and a set of test data.⁴ A test suite, also known as the validation suite, is a collection of various test cases that are grouped (and run together) and intended to test a software product or its specific functionality.⁵ In general, test suites are created following a test plan, and contain both detailed instructions for selected test cases and important information on system configuration to be used during software testing, such as application, environment, version, and so on. It is also worth noting that a test suite consists of functional or non-functional testing and can be used for various test automation tools, such as JUnit, Selenium, and so on.⁶

Software is frequently updated to new versions because of changes in company goals or users' demands for additional functionality. It is essential to re-execute existing test cases and generate extra cases to surface any new faults that affect functionalities. This process is called regression testing.⁷ After several trials of regression testing, the test suite grows in size. Due to resource limitations, testers must reduce the test suite size or select specific test cases. Test suite reduction is generally used to find the minimal test suite subset by determining which test cases to retain or remove. For example, Kalaei and Rafe proposed a strategy for optimal test case generation using cause-effect graphs (CEGs) and utilized the particle swarm optimization (PSO) algorithm to select the optimal test suite.⁵

Previously, considerable research focused on addressing test suite reduction, with numerous approaches proposed.^{8–14} Some heuristic greedy algorithms select test cases for the test suite subset by repeating these two steps: (1) choose a test case that can satisfy the maximum number of unsatisfied requirements or randomly choose one if there are multiple candidates and (2) mark all the requirements that the chosen test case satisfies. Lin and Huang¹¹ reported step (1) as a tie if two or more test cases have the same importance and proposed a tie-breaking technique to solve this problem. Additionally, Lin et al.¹⁵ proposed using Irreplaceability and EIrreplaceability to evaluate the possibility that each test case can be replaced by others during test suite reduction. Their experimental results demonstrated that incorporating the concept of test case irreplaceability into the traditional test suite reduction algorithms leads to representative sets with low execution costs. When there is more than one candidate, some traditional algorithms will randomly select one into the test suite. However, random selection might exclude important test cases with higher fault detection effectiveness.

It is a key issue for testers to use program-based ("white box") testing criteria as a systematic method to justify the integrity of testing.¹⁶ Program-based approaches to testing are generally divided into two types: control flow coverage methods (e.g., branch coverage) and data flow coverage methods (e.g., du-pair coverage).^{8,10,12,14,17,18} For example, branch coverage, also known as decision coverage, is the fraction of the total number of branches that have been executed by the test data.¹⁹ On the other hand, a du-pair is composed of a definition (i.e., assignment) of a variable and a use (i.e., read) of that variable containing an assigned value.²⁰ Du-pair coverage is the fraction of the total number of du-pairs that have been executed by the test data. Hutchins et al.²¹ reported no clear winner between control flow and data flow coverage criteria, after conducting experiments, showing two criteria displayed different characteristics. Fault detection capability can be improved when more than one criterion is considered. One criterion may not be able to test certain blocks of code, whereas another criterion can. To make up for the differences between any two criteria and enhance fault detection, we attempted to combine two criteria using different weighted means, including linear and non-linear combinations. In this way, we can flexibly and simultaneously consider two various types of testing criteria. Furthermore, when more than one

of the test cases are equally important to a suite in the primary criterion (a tie condition), the secondary criterion will help break the tie during the reduction process.

Based on our previous studies,^{11–13,22,23} we select four traditional test suite reduction algorithms as candidate methods to organize the test suite reduction process: the Harrold-Gupta-Soffa (HGS) algorithm presented by Harrold et al.,⁸ the GRE²⁴ and GE algorithms both proposed by Chen and Lau,^{7,24,25} and the Greedy algorithm.²⁶ Furthermore, we modify the means of these traditional algorithms to choose a test case, known as the evaluation metrics. A genetic algorithm (GA) is used to find the best weighting factor assignments for our control flow coverage and data flow coverage criteria. Experiments ranging from 162 LOCs to 9127 LOCs^{21,27} are conducted and analyzed based on eight real subject programs: *tcas*, *schedule2*, *schedule*, *totinfo*, *printtokens2*, *printtokens*, *replace*, and *space*. We evaluate the performance of our proposed approaches with criteria and an extensive comparison with the extant test suite reduction methods. Experimental results reveal our proposed WC-HGS, WC-GRE, WC-GE, and WC-Greedy algorithms result in improved test suite size reduction capability and fault detection effectiveness. For example, the average original test suite size for the selected *schedule2* program is 16.7 test cases; after the test suite reduction process, the average reduced sizes are 4.9 test cases for both the greedy and WH-Greedy algorithms. However, the *Fault Detection Effectiveness Loss (FDE Loss)* of the WH-Greedy algorithm is 13% better than the greedy algorithm. Furthermore, the average original test suite size for the *space* program is 346.6 test cases. After performing the test suite reduction process, the average reduced sizes are 119 and 115 test cases for the greedy and WCU-greedy algorithms, respectively. However, the *FDE Loss* of the WCU-Greedy is 71% better than the greedy algorithm. The WC-Greedy algorithm apparently outperforms the greedy algorithm.

Lastly, we also propose a three-objective binary integer linear programming (ILP) model, which is a linear method to find the minimum subset of the initial test suite that can satisfy all three criteria. A minimum set cover problem can generally be expressed as a binary ILP problem by encoding test-case criteria as objective functions of a binary ILP problem with some given constraints. Hence, the ILP model differs from the traditional algorithms by having both subjective and objective functions. Our experiments demonstrate that although the average reduced test suite size is a little larger than that of traditional methods, we achieve full coverage of all criteria. Using the *printtokens* program as an example, the average reduced size for traditional algorithms is 7.275 test cases and 8.2 for ILP. However, the average *FDE Loss* of the former is approximately 33.75% and 0% for the latter—with all requirements covered. That is, we reduced the test suite size with no loss of fault detection. This study's contributions can be summarized as follows:

- (i) We consider both control flow and the data flow as testing criteria and combine them by using a weighted means with four existing test suite reduction algorithms. A GA is used to assign the best combination of weighting factors to the two testing criteria.
- (ii) To make a fair performance comparison between our proposed methods and some traditional test suite reduction algorithms, we performed experiments based on eight subject programs commonly used in recent studies.
- (iii) We not only consider control flow coverage and data flow coverage as our testing criteria but also further consider the fault detection capability of each test case to enhance the number of exposed faults.
- (iv) A three-objective binary ILP model is proposed.

The remainder of this paper is organized as follows. In Section 2, we briefly review some existing test suite reduction techniques. In Section 3, we introduce some well-known existing algorithms. In Section 4, we further discuss considering multiple structural testing criteria simultaneously, whereas the implementation of our weighted combinatorial algorithms is introduced in Section 5. Then, we explain our weighting factor assignment process using a GA and discuss and analyze the comparison between our proposed approaches and traditional test suite reduction algorithms. Section 6 provides experimental results based on real subject programs that evaluate the performance of our proposed approaches. Threats to internal and external validity are described and discussed in Section 6. In Section 7, we propose a three-objective binary ILP model along with some experiments and discussion. Finally, our conclusions and future work are provided in Section 8.

2 | RELATED WORK

Software testing is commonly used to justify software quality. When software enters the testing phase, developers and/or independent test groups (ITGs) usually must examine it using various testing techniques and test cases to find faults. As software is updated, developers or ITGs must re-execute the original test cases and generate new ones checking if the

software is 100% bug free. As the test process proceeds, the size of the test suite or the set of test cases increases—becoming too large to test all test cases. The test suite may also contain several redundant test cases. This can lead to delays in project release dates. Test suite reduction techniques are well-researched. For example, Huang et al.¹² adopted the fuzzy expert system (FES) approach to enhance fault detection capability; using the approach of partial members in each group to classify test cases. Sampath and Bryce¹⁴ introduced the concept of ordering the test cases in a reduced test suite and enhance the fault detection rate. Kandil et al.¹⁷ proposed an automated agile regression testing approach on both sprints and release levels. Additionally, Heimdahl and George²⁸ conducted an experiment using a large case example of a flight guidance system, which generated reduced test suites for a variety of structural coverage criteria; simultaneously, they preserved coverage and recorded the fault-finding effectiveness. Their results indicated the size of the specification-based test suites was significantly reduced, but the reduced test suites' fault detection was adversely affected.

The Harrold-Gupta-Soffa (HGS) algorithm was originally proposed by Harrold et al.⁸ and has been widely discussed.^{11,12,18,23,26} Although the HGS algorithm was only provided to solve regression testing problems, it has been widely discussed and compared. The HGS algorithm has served as a basic algorithm to derive new reduction algorithms.^{7,10,11,18} The greedy, redundant, and essential (GRE) algorithm was proposed by Chen and Lau.^{12,23,24} Chen and Lau classified test cases into two specified types: essential and one-to-one redundant test cases, where each has been given different priorities in the selection process. Further, the Greedy and Essential (GE) algorithm was also proposed by Chen and Lau.²⁴ Note that the GE and GRE algorithms were also proposed to solve regression testing problems and have demonstrated better or equal performance to the HGS algorithm.²⁴ The greedy algorithm^{12,23,26} is a classical method and the simplest way to find a solution and it is the most well-known traditional algorithm used for a large number of problems to find near-optimal solutions.

Moreover, Mansour and El-Fakih²⁹ applied a hybrid GA to the test suite reduction problem. Zhang et al. reviewed test suite reduction techniques for Java programs using four traditional techniques for Junit test suites. Their empirical results indicated four techniques can effectively reduce Junit test suites, while keeping the same fault detection capability. On the other hand, Rothermel et al.³⁰ noticed that the effectiveness of fault detection can be compromised by test suite reductions. Shi et al.³¹ also proposed an approach that allowed an exploration of trade-offs in test suite reduction; their evolution-aware metrics demonstrated, after performing the test suite reduction, how much the quality of reduced test suites can change. In addition, Di Nardo et al.³² evaluated common prioritization techniques, examining effects of using different coverage criteria on effectiveness. Their results demonstrated prioritization techniques based on additional coverage with finer grained coverage criteria performed better in terms of fault detection rates.

Numerous studies have indicated that reaching full coverage is the termination condition to stop the current testing process.^{8,10–12,18} Hutchins et al.²¹ analyzed the effectiveness of using control flow coverage and data flow coverage. Rothermel et al.³³ selected test cases that mostly satisfied additional statement coverage. Jeffrey and Gupta¹⁸ and Lin and Huang¹¹ used both branch coverage and definition-use pair (du-pair) coverage as their testing criteria. Zhang et al.³⁴ proposed the use of ILP for the test case selection.

Yoo and Harman³⁵ introduced the concept of Pareto efficient multiobjective optimization to the problem of test suite minimization, describing the benefits of Pareto efficient multiobjective optimization. They also reported an empirical study investigating the relative effectiveness of two algorithms for Pareto efficient multiobjective test suite minimization. They used hybrid multiobjective GAs combining the efficient approximation of the greedy approach with the capability of population-based GAs to produce higher-quality Pareto fronts. Their experimental results demonstrated their proposed hybrid approaches outperformed the greedy approach.

In addition, Lin et al.¹⁵ proposed some cost-aware algorithms that produced the representative sets with lower execution costs. Results revealed their algorithms produced representative sets with a low cost to produce a high level of test coverage. Chiu²³ combined some weighting means with traditional test suite reduction techniques and proposed weighted combinatorial algorithms using both control flow and data flow testing criteria to break a tie situation and enhance fault detection effectiveness. Chen²² proposed test suite reduction algorithms using the fuzzy expert system with both control flow and data flow testing criteria to enhance the capability of fault detection.

On the other hand, Eghbali et al.³⁶ combined defect prediction and test case prioritization to break a tie within the process, overcoming the low rate of fault detection caused by ties. They proposed tie breaker approach incorporates an estimate of the fault-proneness of software units—called Supervised Tie Breaker (STB). Their study results indicated using supervision in tie breaking substantially improves the rate of fault detection. Coviello et al.³⁷ investigated a clustering-based approach for inadequate test suite reduction and compared it with well-known adequate approaches like HGS. They defined guidelines for trade-offs between reductions both in test suite size and loss in fault-detection capability for experimental instances to rigorously evaluate cluster-based test suite reduction. Although this research

concluded cluster-based test suite reduction is an inadequate technique, it still can contribute to a more informed decision by providing guidelines of balancing size, coverage, and fault-detection loss of reduced test suites when using clustering. Cruciani et al.³⁸ presented a family of approaches for similarity-based test suite reduction, applying algorithms borrowed from the big data domain together with smart heuristics for finding an evenly spread subset of test cases. Since artifacts usually used in existing test suite reduction techniques, like coverage metrics or specifications, are not always available in a large system, the authors instead use test case text as input. That study concluded the fault detection loss of their approaches was comparable to state-of-the-art techniques while providing high efficiency.

From the mentioned studies, we can observe using two criteria improves fault detection effectiveness. Furthermore, the way to determine proper weighting factors is also a crucial issue.³⁷ GAs^{29,38} are often used to find a near optimal solution to a problem. To find the best combination of two criteria, the evolution of the weighting factor assignment should be convergent. To solve such problems, we use a weighted approach with GAs for test suite reduction.^{8,11,18} It is worth noting that we use branch coverage and du-pair coverage as the testing criteria for software test suite reduction. We attempt to use different kinds of coverage to lower the bias as much as possible.

Practically, testers often use program-based structural testing criteria¹⁶ in a systematic procedure to evaluate whether every part of the program is checked carefully during testing. Traditionally, there are two well-known types of structural testing criteria: *control flow coverage* and *data flow coverage*.³⁹ *Control flow* is where one defines operations and their order of execution. The well-known four kinds of control flow coverage include the following: 1) statement coverage, 2) branch coverage (i.e., structural coverage), 3) path coverage, and 4) conditional coverage.²⁵ Furthermore, several previous studies have only considered statement coverage.^{13,17,35} However, reaching full statement coverage does not address all branches in a program,¹ which means using statement coverage may not be strict enough. Branch coverage is a measure of the percentage of the Boolean expressions of the program that have been invoked and evaluated as true and false in the testing. On the other hand, *data flow* is used to define the data stream, where data sources come from, how data should be transformed, and where data should be loaded. The five kinds of well-known data flow coverage include the following: 1) all-paths coverage, 2) du-pair coverage (or all-uses coverage), all-defs coverage, 4) all-c-uses coverage, and 5) all-p-uses coverage.⁴⁰ Du-pair coverage has been the most commonly used kind of data flow coverage in previous research.^{8,11,18} The du-pair is a pairing of definition and use of a variable with at least one definition-clear path between them.

Numerous studies have only used one type of testing criteria—control flow coverage criterion (e.g., branch coverage) or data flow coverage criterion (e.g., du-pair).^{7,11,25} When there is more than one test case with the same importance, the algorithms mentioned above will randomly select one to break the tie. However, to test a program thoroughly and enhance the effectiveness of fault detection, we consider the *branch coverage* and the *du-pair coverage* as criteria simultaneously in this paper.^{12,26} Two kinds of code-based testing criteria were used to prevent the tie condition during the minimization process.

3 | WEIGHTED COMBINATORIAL METHODS

3.1 | Weighted statistical means

In the past, some studies have indicated that the HGS, GRE, GE, and greedy algorithms randomly chose one test case into the test suite when facing a tie.^{10,41} However, random selection might exclude important test cases that have higher fault detection effectiveness. To avoid this scenario, we propose adopting the two types of criteria simultaneously of control flow coverage and data flow coverage. In this way, we not only prevent our approach from losing important test cases with higher fault detection effectiveness but also break the tie situation.¹¹ Here, we propose to use some weighted statistical means to combine the two types of testing criteria. The main reason for selecting multiple statistical means is to avoid bias and determine the best method of reducing our test suite size. Traditionally, the statistical means has served as a classical method to derive the central tendency of data distributions.⁴² The five selected statistical means are as follows: *weighted arithmetic*, *weighted geometric*, *weighted harmonic*, *weighted contraharmonic*, and *weighted cubic means*.^{13,23} Generally, these weighted means can be described as quasi-arithmetic means (based on the transformation f) weighted by w , given by the following expression:

$$M_w^f(a_1, \dots, a_n) = f^{-1} \left(\frac{\sum_{i=1}^n w_i \cdot f(a_i)}{\sum_{i=1}^n w_i} \right). \quad (1)$$

Note that quasi-arithmetic means correspond to weighted arithmetic means with $f(x) = x$, weighted geometric means with $f(x) = \log(x)$, weighted harmonic means with $f(x) = 1/x$, and weighted cubic means with $f(x) = x^3$, respectively.⁴³

- I. The *Weighted Arithmetic Mean* (WA): WA is normally used in geometry, economics, sociology, history, and so on. Here WA is defined by the following:

$$m_{WA} = \sum_{i=1}^n (w_i \times m_i), \sum_{i=1}^n w_i = 1, 0 \leq w_i \leq 1. \quad (2)$$

where m_i is a non-empty set of data and w_i is a set of non-negative weights that is normalized.

- I. The *Weighted Geometric Mean* (WG): WG is usually used in geometry, finance, and social science. Here, WG is defined as follows:

$$m_{WG} = \prod_{i=1}^n (m_i^{w_i}), \sum_{i=1}^n w_i = 1, 0 \leq w_i \leq 1. \quad (3)$$

- II. The *Weighted Harmonic Mean* (WH): WH is typically used in physics, computer science, electronics, finance, and geometry. Here, WH is defined by the following:

$$m_{WH} = \frac{1}{\sum_{i=1}^n \left(w_i \times \frac{1}{m_i} \right)}, \sum_{i=1}^n w_i = 1, 0 \leq w_i \leq 1. \quad (4)$$

- III. The *Weighted Contraharmonic Mean* (WCH): WCH is commonly used in statistics. Here WCH is defined as follows:

$$m_{WCH} = \frac{\sum_{i=1}^n w_i \times m_i^2}{\sum_{i=1}^n m_i}, \sum_{i=1}^n w_i = 1, 0 \leq w_i \leq 1. \quad (5)$$

- IV. The *Weighted Cubic Mean* (WCU): WCU is typically used in predicting the life expectancy of machine parts. Here, WCU is defined by the following expression:

$$m_{WCU} = \sqrt[3]{\frac{\sum_{i=1}^n w_i \times m_i^3}{n}}, \sum_{i=1}^n w_i = 1, 0 \leq w_i \leq 1. \quad (6)$$

It should be noted that in Equations (2)–(6), for our selected approaches, parameters m_1 , m_2 , w_1 , and w_2 represent the original set of branch coverage contribution, the original set of du-pair coverage contribution, weighting factor assignment for branch coverage, and weighting factor assignment for du-pair coverage, respectively. It is essential to properly assign values to each term to apply these weighted means in a test suite reduction strategy. The branch and du-pair coverage information are extracted by manually inserting a corresponding flag in the position where a branch or du-pair occurs. According to the coverage information, we find the total number of du-pairs can be larger than that of the branches. For example, in one of our test programs, *totinfo*, there are 44 branches and 146 du-pairs. Hence, a test case may satisfy more than 44 uncovered du-pairs, whereas at most 44 uncovered branches are satisfied at the same time. If we define the summation of the number of satisfied branches and du-pairs as our objective, branch coverage turns out to be not as important as du-pair coverage. Consequently, it may bias the test case selection against branch coverage.

In order to prevent one criterion from dominating the entire evaluation metrics, we plan to normalize both control flow and data flow coverage. Thus, we can obtain the covered requirement percentage for these two criteria, which can be used in our evaluation metrics. It is also worth noting that Yoo and Harman⁷ balanced the trade-off between two and three objectives, which performed more efficient testing decisions in some test case prioritization problems. We take advantage

of such an approach in Equation (7)³⁵:

$$f' = \sum_{i=1}^n (w_i \times f_i), \sum_{i=1}^n w_i = 1. \quad (7)$$

Experimental results demonstrated that their optimization process performed well by implementing an equally-weighted objective function.

3.2 | Evaluation metric

Jones and Harrold⁴⁴ considered the contribution of each test case in the test suite reduction algorithm. Accordingly, a test case with a higher contribution to the original test suite T was chosen. The formula is defined as follows:

$$ContributionToSuite(t) = \sum_{i=1}^n Contribution(t, r_i). \quad (8)$$

Here, Set $R = \{r_1, r_2, r_3, \dots, r_n\}$ contains all test requirements, and r_i is the i^{th} requirement in set R . Notice that t represents a test case in the test suite T . Furthermore,

$$Contribution(t, r_i) = \begin{cases} 0, & \text{if } t \text{ can't satisfy } r_i, \\ \frac{1}{(\text{the number of test cases that satisfy } r_i)}, & \text{otherwise} \end{cases}. \quad (9)$$

After that, Lin et al.¹² derived their metric, *Eirreplaceability*, from Equations (8) and (9). They first selected essential test cases to avoid, including more redundant test cases, which proved to have better size reduction capability, as illustrated in Equation (10):

$$EIrreplaceability(t, r_i) = \begin{cases} \infty, & \exists i, 1 \leq i \leq n, r_i \text{ can be satisfied by } t \text{ only,} \\ \sum_{i=1}^n Contribution(t, r_i), & \text{otherwise} \end{cases} \quad (10)$$

However, to evaluate the coverage percentage of a test case, we need to further consider the uncovered branches and du-pairs of the remaining test cases. That is, we will evaluate the maximum contribution, assuming the test case is able to satisfy all uncovered requirements. Here, we define this metric called *MaxContribution*(t, r_i), as follows:

$$MaxContribution(t, r_i) = \begin{cases} \frac{1}{(\text{the number of test cases that satisfy } r_i) + 1}, & \text{if } t \text{ can't satisfy } r_i, \\ \frac{1}{(\text{the number of test cases that satisfy } r_i)}, & \text{otherwise} \end{cases}. \quad (11)$$

Then, we propose the *Weighted Combinatorial Eirreplaceability* (WCEI). It is evaluated as a fraction to represent the percentage of branch coverage and du-pair coverage by the following:

$$WCEI(t, r_i) = \begin{cases} \infty, & \exists i, 1 \leq i \leq n, r_i \text{ can be satisfied by } t \text{ only,} \\ \frac{\sum_{i=1}^n Contribution(t, r_i)}{\sum_{i=1}^n MaxContribution(t, r_i)}, & \text{otherwise.} \end{cases} \quad (12)$$

Note that the pseudo code of the function *EvaluateWCEI*() is provided in Figure 1. Testers need to select a test case to be evaluated and input the target test case list and the corresponding relationship information. After that, from lines 17 to 37, $\sum_{i=1}^n Contribution(t, r_i)$ and $\sum_{i=1}^n MaxContribution(t, r_i)$ for branch coverage and du-pair coverage are evaluated separately. Finally, from lines 39 to 54, if the selected test case is not essential and is able to satisfy at least one branch

```

1 function EvaluateWCEI (t, TSList,  $S^c$ ,  $S^d$ , wm)
2
3 input
4 t: a test case chosen to evaluate WCEI
5 TSList: the list containing the considered test cases
6  $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S_i^c$  contains test cases in TSList which satisfy control flow requirement  $c_i$ 
7  $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S_i^d$  contains test cases in TSList which satisfy data flow requirement  $d_i$ 
8 wm: the weighted mean chosen for combination
9 output
10 WCEI: the evaluated WCEI value of the selected test case
11
12 begin
13 Contribution_C  $\leftarrow$  0; MaxContribution_C  $\leftarrow$  0;
14 Contribution_D  $\leftarrow$  0; MaxContribution_D  $\leftarrow$  0;
15 WCEI_C  $\leftarrow$  0; WCEI_D  $\leftarrow$  0; WCEI  $\leftarrow$  0; Essential  $\leftarrow$  false;
16 for-each ( $c_i$  is not marked)
17   if (t satisfies  $c_i$ )
18     if ( $|S_i^c| = 1$ )
19       Essential  $\leftarrow$  true; // t is an essential test case to satisfy  $c_i$ .
20     else
21       Contribution_C  $\leftarrow$  Contribution_C + 1 /  $|S_i^c|$ ;
22       MaxContribution_C  $\leftarrow$  MaxContribution_C + 1 /  $|S_i^c|$ ;
23     end-if
24   else // t can't satisfy  $c_i$ 
25     MaxContribution_C  $\leftarrow$  MaxContribution_C + 1 / ( $|S_i^c|$  + 1);
26   end-if
27 end-for
28
29 for-each ( $d_i$  is not marked)
30   if (t satisfies  $d_i$ )
31     Contribution_D  $\leftarrow$  Contribution_D + 1 /  $|S_i^d|$ ;
32     MaxContribution_D  $\leftarrow$  MaxContribution_D + 1 /  $|S_i^d|$ ;
33   else // t can't satisfy  $d_i$ 
34     MaxContribution_D  $\leftarrow$  MaxContribution_D + 1 / ( $|S_i^d|$  + 1);
35   end-if
36 end-for
37
38 if (Essential = true)
39   WCEI  $\leftarrow$   $\infty$ ;
40 else
41   if (t can't satisfy any  $c_i$  or  $d_i$ )
42     WCEI  $\leftarrow$  0;
43   else
44     WCEI_C  $\leftarrow$   $\frac{\text{Contribution\_C}}{\text{MaxContribution\_C}}$ ; WCEI_D  $\leftarrow$   $\frac{\text{Contribution\_D}}{\text{MaxContribution\_D}}$ ;
45     switch (wm)
46       case (WA): WCEI  $\leftarrow$   $w_c \times \text{WCEI\_C} + w_d \times \text{WCEI\_D}$ ;
47       case (WG): WCEI  $\leftarrow$   $\text{WCEI\_C}^{w_c} \times \text{WCEI\_D}^{w_d}$ ;
48       case (WH): WCEI  $\leftarrow$   $\frac{1}{(w_c \times \frac{1}{\text{WCEI\_C}} + w_d \times \frac{1}{\text{WCEI\_D}})}$ ;
49       case (WCH): WCEI  $\leftarrow$   $\frac{w_c \times \text{WCEI\_C}^2 + w_d \times \text{WCEI\_D}^2}{w_c \times \text{WCEI\_C} + w_d \times \text{WCEI\_D}}$ ;
50       case (WCu): WCEI  $\leftarrow$   $\sqrt[3]{\frac{w_c \times \text{WCEI\_C}^3 + w_d \times \text{WCEI\_D}^3}{2}}$ ;
51     end-switch
52   end-if
53 end-if
54 return WCEI;
55 end

```

FIGURE 1 The pseudo code of function EvaluateWCEI()

or du-pair, then one of the five proposed weighted means is implemented. Furthermore, $WCEI(t)$ will be evaluated using Equation (12).

4 | PROPOSED WEIGHTED COMBINATORIAL ALGORITHMS

In this section, we demonstrate how to modify four well-known traditional algorithms and adopt the evaluation metrics with different weighted means into the algorithms. In our proposed approach, we set our termination condition as reaching the full coverage of the branch to stop the reduction process.¹⁸

4.1 | WC-HGS: The weighted combinatorial HGS algorithm

The pseudo code of the whole weighted combinatorial HGS algorithm,²³ hereafter called the WC-HGS algorithm, is presented in Figure 2. Note that we modify the evaluation function to consider two testing criteria simultaneously and provide an additional and selective input of the parameter wm for testers to select the weighted mean. As a result, our proposed WC-HGS algorithm selects test cases with a higher du-pair coverage percentage. When all control flow requirements are satisfied, test cases in RS are able to reach near full coverage of both testing criteria, which leads to enhanced fault detection effectiveness.

4.2 | WC-GRE: The weighted combinatorial GRE algorithm

The pseudo code of the WC-GRE is depicted in Figure 3.²³ Additionally, Figure 4 depicts the sub-function *RemoveRedundant()* to implement the one-to-one redundant strategy. Note that here we still modify the evaluation function to consider two testing criteria at the same time. Furthermore, we also provide an additional and selective input of parameter wm for testers to select the weighted mean method according to their requirements. As a result, the proposed WC-GRE algorithm selects test cases with a better du-pair coverage percentage. When all control flow requirements are satisfied, test cases in RS are able to reach almost full coverage of both testing criteria, which leads to enhanced fault detection effectiveness.

4.3 | WC-GE: The weighted combinatorial GE algorithm

The pseudo code of the weighted combinatorial GE (WC-GE) algorithm is provided in Figure 5. Similarly, in order to consider two testing criteria simultaneously, we modify the evaluation function. We also provide an additional input for parameter wm to select the weighted mean method according to testers' requirements. As a result, our proposed WC-GE algorithm selects test cases with a better du-pair coverage percentage. When all control flow requirements are satisfied, test cases in RS reach almost full coverage of both testing criteria, meaning fault detection effectiveness is enhanced.

4.4 | WC-greedy: The weighted combinatorial greedy algorithm

Figure 6 depicts the pseudo code of the weighted combinatorial greedy algorithm (WC-Greedy).²³ We modify the evaluation function from considering just one primary testing criterion to considering two testing criteria simultaneously. Furthermore, we provide an additional input for parameter wm to select the weighted mean according to the testers' requirements. As a result, our proposed WC-Greedy algorithm selects test cases with a better du-pair coverage percentage. When all control flow requirements are satisfied, test cases in RS are able to reach near full coverage of both testing criteria, leading to enhanced fault detection effectiveness.

In this paper, GA is used to find the best weighting factor assignments for our control flow coverage and data flow coverage criteria. GAs are frequently used to deal with optimization problems. Traditionally, solution populations in GA are called chromosomes. GAs simulate the progress of chromosome generation, and chromosomes can be made up of binary strings or of real values. Binary-coded GAs have chromosomes of 0s and 1s. These are more traditional methods and are adequate for small- to moderate-size optimization problems. Real-coded GAs utilize real values that allow for

FIGURE 2 The pseudo code of WC-HGS

```

1 algorithm WC-GRE
2
3 input
4    $TS$ : original generated (unreduced) test suite
5    $C$ : set of all control flow requirements
6    $D$ : set of all data flow requirements
7    $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S_i^c$  contains test cases which satisfy control flow requirement  $c_i$ 
8    $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S_i^d$  contains test cases which satisfy data flow requirement  $d_i$ 
9    $R^c = \{R_1^c, R_2^c, \dots, R_n^c\}$ :  $R_i^c$  contains control flow requirements which are satisfied by  $t_i$ 
10   $R^d = \{R_1^d, R_2^d, \dots, R_m^d\}$ :  $R_i^d$  contains data flow requirements which are satisfied by  $t_i$ 
11 output
12  $RS$ : a representative reduced set of  $TS$ 
13 declare
14  $Unsatisfied\_C$ : set of unsatisfied control flow requirements
15  $Unsatisfied\_D$ : set of unsatisfied data flow requirements
16  $RSTemp$ : selected set of test cases in each loop
17  $TS'$ : set of unselected and non 1-to-1 redundant test cases
18
19 begin
20  $RS \leftarrow \{\}$ ;  $Unsatisfied\_C \leftarrow C$ ;  $TS' \leftarrow TS$ ; // initialization
21  $RS \leftarrow \bigcup_i t_i$ , for  $c_i \in C$ ,  $t_i \in |S_i^c|$ ,  $|S_i^c| = 1$ ; // (1) essential strategy
22  $Unsatisfied\_C \leftarrow C - (\bigcup_j S_j^c, t_j \in RS)$ ;  $TS' = TS' - RS$ ;
23 for-each  $t_i \in (TS' - RS)$  do
24    $R_i^c \leftarrow R_i^c - (\bigcup_j R_j^c, t_j \in RS)$ ;
25    $R_i^d \leftarrow R_i^d - (\bigcup_j R_j^d, t_j \in RS)$ ;
26 end-for
27 for-each  $t_i \in RS$  do
28    $R_i^c \leftarrow \{\}$ ;  $R_i^d \leftarrow \{\}$ ;
29 end-for
30 for-each  $d_i \in Unsatisfied\_D$  do  $S_i^d \leftarrow S_i^d - RS$ ;
31 for-each  $d_i \notin Unsatisfied\_D$  do  $S_i^d \leftarrow \{\}$ ;
32 while ( $Unsatisfied\_C$  is not empty)
33    $RSTemp \leftarrow \{\}$ ;
34   // (2) 1-to-1 redundant strategy
35    $RemoveRedundant(TS', R_1^c, \dots, R_n^c, R_1^d, \dots, R_m^d, S_1^c, \dots, S_n^c, S_1^d, \dots, S_m^d)$ ;
36   for each (test case  $t$  in  $TS'$  with 1-to-1 redundant test cases removed) do
37      $WCEI[t] \leftarrow EvaluateWCEI(t, TS', S^c, S^d, wm)$ ;
38   end for
39   if (there are  $c_i$ 's which  $|S_i^c| = 1$ )
40      $RSTemp \leftarrow (\bigcup_j t_j, t_j \in S_i^c, c_i \in Unsatisfied\_C)$  // (1) essential strategy
41   else
42      $RSTemp \leftarrow t$  with maximum  $WCEI[t]$ ; // (3) greedy strategy
43   end-if
44    $RS \leftarrow RS \cup RSTemp$ ;
45    $TS' \leftarrow TS' - RSTemp$ ;
46    $Unsatisfied\_C \leftarrow Unsatisfied\_C - (\bigcup_j R_j^c, t_j \in RSTemp)$ ;
47    $Unsatisfied\_D \leftarrow Unsatisfied\_D - (\bigcup_j R_j^d, t_j \in RSTemp)$ ;
48   for-each  $c_i \in Unsatisfied\_C$  do  $S_i^c \leftarrow S_i^c - RSTemp$ ;
49   for-each  $c_i \notin Unsatisfied\_C$  do  $S_i^c \leftarrow \{\}$ ;
50   for-each  $d_i \in Unsatisfied\_D$  do  $S_i^d \leftarrow S_i^d - RSTemp$ ;
51   for-each  $d_i \notin Unsatisfied\_D$  do  $S_i^d \leftarrow \{\}$ ;
52   for-each  $t_i \in TS'$  do
53      $R_i^c \leftarrow R_i^c - (\bigcup_j R_j^c, t_j \in RSTemp)$ ;
54      $R_i^d \leftarrow R_i^d - (\bigcup_j R_j^d, t_j \in RSTemp)$ ;
55   end-for
56   for-each  $t_i \in RSTemp$  do
57      $R_i^c \leftarrow \{\}$ ;  $R_i^d \leftarrow \{\}$ ;
58   end-for
59 end-while
60 return  $RS$ ;
61 end

```

improving computational memory and time, compared to binary-coded GAs. Mitchell³⁶ once argued that much of the existing GA theory is based on the assumption of fixed-length and fixed-order binary encodings, and much of that theory can be extended to apply to non-binary encodings. Mitchell³⁶ also mentioned that some empirical comparisons between binary encodings and real-valued encodings have demonstrated better performance for the latter; however, the performance depends mainly on the problem and the details of the GA being used. Real-coded GA is faster than

```

1 algorithm WC-GRE
2
3 input
4  $TS$ : original generated (unreduced) test suite
5  $C$ : set of all control flow requirements
6  $D$ : set of all data flow requirements
7  $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S_i^c$  contains test cases which satisfy control flow requirement  $c_i$ 
8  $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S_i^d$  contains test cases which satisfy data flow requirement  $d_i$ 
9  $R^c = \{R_1^c, R_2^c, \dots, R_n^c\}$ :  $R_i^c$  contains control flow requirements which are satisfied by  $t_i$ 
10  $R^d = \{R_1^d, R_2^d, \dots, R_m^d\}$ :  $R_i^d$  contains data flow requirements which are satisfied by  $t_i$ 
11 output
12  $RS$ : a representative reduced set of  $TS$ 
13 declare
14  $Unsatisfied\_C$ : set of unsatisfied control flow requirements
15  $Unsatisfied\_D$ : set of unsatisfied data flow requirements
16  $RSTemp$ : selected set of test cases in each loop
17  $TS'$ : set of unselected and non 1-to-1 redundant test cases
18
19 begin
20  $RS \leftarrow \{\}$ ;  $Unsatisfied\_C \leftarrow C$ ;  $TS' \leftarrow TS$ ; // initialization
21  $RS \leftarrow \bigcup_i t_i$ , for  $c_i \in C$ ,  $t_i \in |S_i^c|$ ,  $|S_i^c| = 1$ ; // (1) essential strategy
22  $Unsatisfied\_C \leftarrow C - (\bigcup_j S_j^c, t_j \in RS)$ ;  $TS' = TS' - RS$ ;
23 for-each  $t_i \in (TS' - RS)$  do
24    $R_i^c \leftarrow R_i^c - (\bigcup_j R_j^c, t_j \in RS)$ ;
25    $R_i^d \leftarrow R_i^d - (\bigcup_j R_j^d, t_j \in RS)$ ;
26 end-for
27 for-each  $t_i \in RS$  do
28    $R_i^c \leftarrow \{\}$ ;  $R_i^d \leftarrow \{\}$ ;
29 end-for
30 for-each  $d_i \in Unsatisfied\_D$  do  $S_i^d \leftarrow S_i^d - RS$ ;
31 for-each  $d_i \notin Unsatisfied\_D$  do  $S_i^d \leftarrow \{\}$ ;
32 while ( $Unsatisfied\_C$  is not empty)
33    $RSTemp \leftarrow \{\}$ ;
34   // (2) 1-to-1 redundant strategy
35    $RemoveRedundant(TS', R_1^c, \dots, R_n^c, R_1^d, \dots, R_m^d, S_1^c, \dots, S_n^c, S_1^d, \dots, S_m^d)$ ;
36   for each (test case  $t$  in  $TS'$  with 1-to-1 redundant test cases removed) do
37      $WCEI[t] \leftarrow EvaluateWCEI(t, TS', S^c, S^d, wm)$ ;
38   end for
39   if (there are  $c_i$ 's which  $|S_i^c| = 1$ )
40      $RSTemp \leftarrow (\bigcup_j t_j, t_j \in S_i^c, c_i \in Unsatisfied\_C)$  // (1) essential strategy
41   else
42      $RSTemp \leftarrow t$  with maximum  $WCEI[t]$ ; // (3) greedy strategy
43   end-if
44    $RS \leftarrow RS \cup RSTemp$ ;
45    $TS' \leftarrow TS' - RSTemp$ ;
46    $Unsatisfied\_C \leftarrow Unsatisfied\_C - (\bigcup_j R_j^c, t_j \in RSTemp)$ ;
47    $Unsatisfied\_D \leftarrow Unsatisfied\_D - (\bigcup_j R_j^d, t_j \in RSTemp)$ ;
48   for-each  $c_i \in Unsatisfied\_C$  do  $S_i^c \leftarrow S_i^c - RSTemp$ ;
49   for-each  $c_i \notin Unsatisfied\_C$  do  $S_i^c \leftarrow \{\}$ ;
50   for-each  $d_i \in Unsatisfied\_D$  do  $S_i^d \leftarrow S_i^d - RSTemp$ ;
51   for-each  $d_i \notin Unsatisfied\_D$  do  $S_i^d \leftarrow \{\}$ ;
52   for-each  $t_i \in TS'$  do
53      $R_i^c \leftarrow R_i^c - (\bigcup_j R_j^c, t_j \in RSTemp)$ ;
54      $R_i^d \leftarrow R_i^d - (\bigcup_j R_j^d, t_j \in RSTemp)$ ;
55   end-for
56   for-each  $t_i \in RSTemp$  do
57      $R_i^c \leftarrow \{\}$ ;  $R_i^d \leftarrow \{\}$ ;
58   end-for
59 end-while
60 return  $RS$ ;
61 end

```

FIGURE 3 The pseudo code of WC-GRE

the binary-coded GA because the chromosomes do not need to be decoded before testing their fitness with the fitness function.⁴⁵ However, it can also be argued that real-coded GA is vulnerable to premature convergence, especially for complex real-world problems with numerous design variables.⁴⁶

There are practically no rigorous guidelines and/or suggestions for predicting which encoding will work best.³⁷ This depends on the solved problem. Thus, deciding on whether to plan a binary-coded GA or real-coded GA or planning

```

1 function RemoveRedundant ( $TS', R_1^c, \dots, R_n^c, R_1^d, \dots, R_m^d, S_1^c, \dots, S_n^c, S_1^d, \dots, S_m^d$ )
2
3 input
4  $TS'$ : set of unselected and non 1-to-1 redundant test cases
5  $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S^c$  contains test cases which satisfy control flow requirement  $c_i$ 
6  $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S^d$  contains test cases which satisfy data flow requirement  $d_i$ 
7  $R^c = \{R_1^c, R_2^c, \dots, R_n^c\}$ :  $R^c$  contains control flow requirements which are satisfied by  $t_i$ 
8  $R^d = \{R_1^d, R_2^d, \dots, R_m^d\}$ :  $R^d$  contains data flow requirements which are satisfied by  $t_i$ 
9 output
10  $TS'$ : the set of test cases with 1-to-1 redundant test cases removed.
11  $S^c$ :  $S^c$  contains test cases which satisfy control flow requirement  $c_i$  with 1-to-1 redundant test cases removed
12  $S^d$ :  $S^d$  contains test cases which satisfy data flow requirement  $d_i$  with 1-to-1 redundant test cases removed
13 declare
14  $\Delta Redundant$ : set of 1-to-1 redundant test cases
15  $size$ : the amount of test cases in input  $TS'$ 
16
17 begin
18 Sort( $TS'$ ); // sort test cases in  $TS$  in descending order of  $|S_i^c|$ 
19  $\Delta Redundant \leftarrow \{\}$ ;
20  $size = |TS'|$ ;
21 for  $i = 1$  to  $(size-1)$  do
22   if ( $t_i \in TS'$ )
23     for  $j = i+1$  to  $size$  do
24       if ( $t_j \in TS'$ )
25         if ( $R_j^c \subseteq R_i^c$ ) // check 1-to-1 redundancy
26            $\Delta Redundant \leftarrow \Delta Redundant \cup \{t_j\}$ ;
27            $TS' \leftarrow TS' - \{t_j\}$ ;
28         end-if
29       end-if
30     end-for
31   end-if
32 end-for
33 for-each  $S_i^c$  do  $S_i^c \leftarrow S_i^c - \Delta Redundant$ ; // remove 1-to-1 redundant test cases
34 for-each  $S_i^d$  do  $S_i^d \leftarrow S_i^d - \Delta Redundant$ ;
35 end

```

FIGURE 4 The pseudo code of function RemoveRedundant()

on how to implement them for a specific problem or application is a separate research topic. In fact, both binary-coded and real-coded GAs have their own niche applications. In this paper, a chromosome in our proposed approach is the weighting factor that is assigned to the control flow requirement. Weighting factors take the form a string of bits with a binary representation. Accordingly, the weighting factor assigned for data flow requirement should be $w_d = 1 - w_c$, $w_d > 0$, $w_c > 0$, where w_d is the weighting factor for data flow requirement, and w_c is the weighting factor for control flow requirement. Note that binary-coded GA, which performed best in continuous optimization problems, is adopted in our study.⁴⁷ The pseudo code of GA is provided in Figure 7. As GA starts, each chromosome is generated randomly and converted into decimal representation. Next, the evolution process occurs, where genetic operators are used to evolve the initial populations. The fitness function is calculated, and the population in the top ranking of the fitness function is selected as parents for crossover and mutation to generate better chromosomes. After the evolution occurs several times, a near optimal solution is obtained as soon as one of the termination conditions are reached.

Considering the selection scheme in GA, the most important component is the definition of the fitness function,⁴⁸ which is the evaluation scheme to find solutions according to the user's objective. A fitness function is an indicator of the fitness of every chromosome. The chromosome with a higher fitness value will have a better chance of being preserved in the next population. In this study, we need to find the representative test suite subset. Therefore, we propose to use the faults-to-test ratio as our fitness function considering size reduction and fault detection effectiveness. The higher the FTT ratio, the more effective the chosen test cases are. FTT ratio can be defined as follows:

$$FTT \text{ ratio} = \frac{\text{the number of faults found in RS}}{\text{the number of test cases in RS}}. \quad (13)$$

```

1 algorithm WC-GE
2
3 input
4    $TS$ : original generated (unreduced) test suite
5    $C$ : set of all control flow requirements
6    $D$ : set of all data flow requirements
7    $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S_i^c$  contains test cases which satisfy control flow requirement  $c_i$ 
8    $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S_i^d$  contains test cases which satisfy data flow requirement  $d_i$ 
9    $R^c = \{R_1^c, R_2^c, \dots, R_n^c\}$ :  $R_i^c$  contains control flow requirements which are satisfied by  $t_i$ 
10   $R^d = \{R_1^d, R_2^d, \dots, R_m^d\}$ :  $R_i^d$  contains data flow requirements which are satisfied by  $t_i$ 
11 output
12  $RS$ : a representative reduced set of  $T$ 
13 declare
14    $Unsatisfied\_C$ : set of unsatisfied control flow requirements
15    $Unsatisfied\_D$ : set of unsatisfied data flow requirements
16    $RSTemp$ : selected set of test cases in each loop
17    $TS'$ : set of unselected test cases
18
19 begin
20  $RS \leftarrow \{\}$ ;  $Unsatisfied\_C \leftarrow C$ ;  $TS' \leftarrow TS$ ; // initialization
21  $RS \leftarrow \bigcup_i t_i$ , for  $c_i \in C$ ,  $t_i \in |S_i^c|$ ,  $|S_i^c| = 1$ ; // Essential Strategy
22  $Unsatisfied\_C \leftarrow C - (\bigcup_j S_j^c, t_j \in RS)$ ;
23  $TS' = TS' - RS$ ;
24 for-each  $t_i \in (TS - RS)$  do
25    $R_i^c \leftarrow R_i^c - (\bigcup_j R_j^c, t_j \in RS)$ ;
26    $R_i^d \leftarrow R_i^d - (\bigcup_j R_j^d, t_j \in RS)$ ;
27 end-for
28 for-each  $t_i \in RS$  do
29    $R_i^c \leftarrow \{\}$ ;  $R_i^d \leftarrow \{\}$ ;
30 end-for
31 for-each  $d_i \in Unsatisfied\_D$  do  $S_i^d \leftarrow S_i^d - RS$ ;
32 for-each  $d_i \notin Unsatisfied\_D$  do  $S_i^d \leftarrow \{\}$ ;
33 while ( $Unsatisfied\_C$  is not empty)
34    $RSTemp \leftarrow \{\}$ ;
35   for each (test case  $t \in TS'$ ) do
36      $WCEI[t] \leftarrow EvaluateWCEI(t, TS', S^c, S^d, wm)$ ;
37   end for
38    $RSTemp \leftarrow t$  with maximum  $WCEI[t]$ ; // Greedy Strategy
39    $RS \leftarrow RS \cup RSTemp$ ;
40    $Unsatisfied\_C \leftarrow Unsatisfied\_C - (\bigcup_j R_j^c, t_j \in RSTemp)$ ;
41    $Unsatisfied\_D \leftarrow Unsatisfied\_D - (\bigcup_j R_j^d, t_j \in RSTemp)$ ;
42   for-each  $c_i \in Unsatisfied\_C$  do  $S_i^c \leftarrow S_i^c - RSTemp$ ;
43   for-each  $c_i \notin Unsatisfied\_C$  do  $S_i^c \leftarrow \{\}$ ;
44   for-each  $d_i \in Unsatisfied\_D$  do  $S_i^d \leftarrow S_i^d - RSTemp$ ;
45   for-each  $d_i \notin Unsatisfied\_D$  do  $S_i^d \leftarrow \{\}$ ;
46   for-each  $t_i \in TS$  do
47      $R_i^c \leftarrow R_i^c - (\bigcup_j R_j^c, t_j \in RSTemp)$ ;
48      $R_i^d \leftarrow R_i^d - (\bigcup_j R_j^d, t_j \in RSTemp)$ ;
49   end-for
50   for-each  $t_i \in RSTemp$  do
51      $R_i^c \leftarrow \{\}$ ;  $R_i^d \leftarrow \{\}$ ;
52   end-for
53 end-while
54 return  $RS$ ;
55 end

```

FIGURE 5 The pseudo code of WC-GE


```

1 algorithm WC-Greedy
2
3 input
4    $TS$ : original generated (unreduced) test suite
5    $C$ : set of all control flow requirements
6    $D$ : set of all data flow requirements
7    $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S_i^c$  contains test cases which satisfy control flow requirement  $c_i$ 
8    $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S_i^d$  contains test cases which satisfy data flow requirement  $d_i$ 
9    $w_m$ : the weighted mean chosen for combination
10 output
11  $RS$ : a representative reduced set of  $TS$ 
12
13 begin
14  $RS \leftarrow \{\}$ ;
15  $WeightedCount[t] \leftarrow 0$  for each test case  $t$ ;
16 loop
17   for-each test case  $t$  in generated test suite  $TS$  do
18      $WCEI[t] \leftarrow EvaluateWCEI(t, TS, S^c, S^d, w_m)$ ;
19   end for
20    $List \leftarrow$  test cases which  $WCEI[t]$  is maximum;
21    $RS \leftarrow RS \cup \{ \text{a test case } t \text{ in } List \}$ ;
22    $TS \leftarrow TS - \{ t \}$ 
23    $S^c \leftarrow S^c - (\bigcup_j S_j^c, t \text{ can satisfy } c_j)$ ;
24    $S^d \leftarrow S^d - (\bigcup_j S_j^d, t \text{ can satisfy } d_j)$ ;
25    $C \leftarrow C - (\bigcup_j c_j, t \text{ can satisfy } c_j)$ ;
26    $D \leftarrow D - (\bigcup_j d_j, t \text{ can satisfy } d_j)$ ;
27 until all requirements in set  $C$  are satisfied ( $C$  is empty)
28 return  $RS$ ;
29 end

```

FIGURE 6 The pseudo code of WC-Greedy

The higher the FTT ratio, the better the quality of the selected test cases.¹¹ Thus, in this paper, we use GA with FTT ratio as the fitness function to find test cases with higher quality.

5 | EXPERIMENTS AND ANALYSIS

In this section, we conduct experiments for the proposed four modified traditional algorithms with five weighted means. We then compare our approaches with traditional algorithms using evaluation criteria. Overall comparisons and analyses are also provided.

5.1 | Experimental setup and subject programs

In this paper, eight Siemens suite programs⁴⁹ developed in C language are chosen as the subject programs to evaluate the performance of our approaches. Furthermore, we modified the traditional algorithms in C++ language. Detailed descriptions of Siemens suite programs can be found in Table 1. Note that *tcas* is an aircraft collision avoidance system. *totinfo* computes statistics given input data. Both *schedule* and *schedule2* are priority schedulers. Both *printtokens* and *printtokens2* are lexical analyzers. *replace* performs pattern matching and substitution. *Space* functions as an interpreter for an array definition language (ADL). *Space* reads a file that contains several ADL statements, and checks the contents of the file for adherence to the ADL grammar and to specific consistency rules.⁴⁹ Today, the Siemens Suite programs are widely used to execute in many test suite reduction studies.^{7,8,10–12} Basically, all programs, faulty versions, and test pools used in our experiments are available from the Software-artifact Infrastructure Repository (SIR) website.⁴⁹ From Table 1, can see seven of eight subject programs contain seeded faults, and the last contains real faults.²¹

```

1 algorithm GA
2
3 declare
4 BinaryRandomGeneration(): to generate random binary strings as inputs to initial populations
5 TSRProcess(): test suite reduction process, which will generate a FTTRatio value output
6 UpdateBestGene(): record and update the best gene found by far
7 RouletteWheelSelection(): Roulette Wheel Selection strategy to do chromosome reproduction
8 SinglePointCrossover(): Single Point Crossover strategy to do gene crossover
9 SinglePointMutation(): Single Point Mutation strategy to mutate the genes in current populations
10
11 begin
12 // Step 1: Population Generation
13 OldPopulation  $\leftarrow$  BinaryRandomGeneration()
14
15 // Step 2: Fitness Value (FTTRatio) Calculation
16 for-each IterationCount do
17
18   FTTRatio  $\leftarrow$  TSRProcess(WeightingFactor);
19   UpdateBestGene();
20
21 // Step 3: Reproduction
22 Population  $\leftarrow$  RouletteWheelSelection();
23
24 // Step 4: Crossover
25 while ( counter < PopulationCount ) do
26   SinglePointCrossover(Population[p], Population[p+1], Pcrossover);
27   Count++;
28 end-while
29
30 // Step 5: Mutation
31 for-each Population do
32   SinglePointMutation(RandPos, Pmutation);
33 end-for
34
35 end-for
36
37 BestFit  $\leftarrow$  the best weighting factor recorded in UpdateBestGene();
38 end

```

FIGURE 7 The pseudo code of GA

TABLE 1 Information of Siemens suite programs

Name	Language	LOC	Faulty Counts	Fault Type	Test Pool Size	Descriptions
tcas	C	162	41	Seeded	1608	Program used to avoid an aircraft collision
totinfo	C	346	23	Seeded	1052	Compute statistics given input data
schedule	C	299	9	Seeded	2650	Process priority scheduler
schedule	C	287	10	Seeded	2710	Process priority scheduler
printtokens	C	378	7	Seeded	4130	Lexical analyzers
printtokens2	C	366	10	Seeded	4115	Lexical analyzers
replace	C	514	32	Seeded	5542	Pattern matching and substitution system
space	C	9127	38	Real	13585	An interpreter for an array definition language

To obtain sufficient test suites for each program, we follow a similar series of steps from the experimental setup proposed by Rothermel et al.,⁵⁰ and we select a subset of test cases from the test pool to form an initial test suite. We first randomly select varying numbers of test cases from the associated test pool, add them to the test suite, and then analyze the branch coverage of the selected test case in the test suite. If these selected test cases in the test suite fail to cover all requirements, more test cases will be added to the test suite to achieve 100% branch coverage. To allow different levels of redundancy, the random amount of test cases we initially add to each suite vary in size from 0 to 0.5 times the number of LOCs in the program. For example, as we can see on Table 1 the LOC of program *tcas* is 162, meaning the range of the *tcas* initial

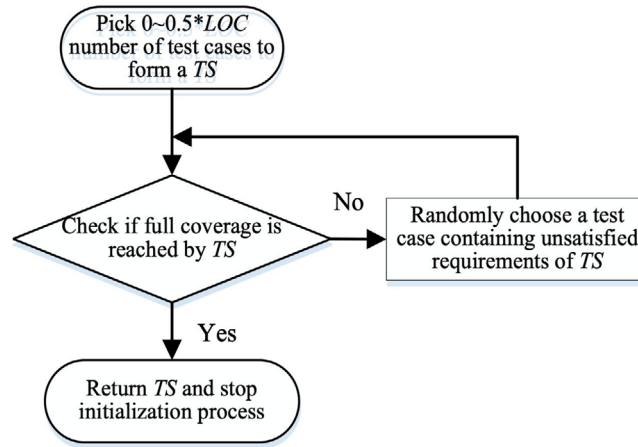


FIGURE 8 The flowchart of the initialization process

test suite size is 0 to 81. Next, we check whether the test cases in a test suite can satisfy all the test requirements (branch coverage and du-pair coverage). The initialization process halts when all test requirements, including branch coverage and du-pair coverage, are satisfied. That is, the initialization process halts only if the default test suite satisfies all 16 branches and 41 du-pairs when there are 16 branches and 41 du-pairs in *tcas* program. Otherwise, we need to randomly select one more test case that can satisfy one or more unsatisfied test requirements from remaining test cases in the test case pool and add the test case(s) into the test suite.¹⁸ We repeat the random selection of test cases until all test requirements are satisfied. We generated 1000 test suites in total for each program. The flow chart of the initialization process is depicted in Figure 8.

5.2 | Comparison criteria

To check the performance of our proposed approaches and make a fairly comprehensive comparison with other traditional test suite reduction techniques while also avoiding bias, we use some objective criteria in this paper. Detailed information is described in the following.

- (i) The *Suite Size Reduction* (SSR) can be defined as^{10,12,18,32}:

$$SSR = \frac{|T| - |T_{\min}|}{|T|} \times 100\%, \quad (14)$$

where $|T|$ is the number of test cases in an initial test suite, and $|T_{\min}|$ is the number of test cases in a reduced test suite subset. Intuitively, if the difference between $|T|$ and $|T_{\min}|$ is significant, SSR is large. The higher the SSR of the test suite subset that is evaluated, the better the suite size reduction that the algorithm makes.

- (i) The *Fault Detection Effectiveness Loss* (FDE Loss) is defined by^{10,12,30}:

$$FDELoss = \frac{|F| - |F_{\min}|}{|F|} \times 100\%, \quad (15)$$

where $|F|$ is the number of faults exposed by test cases in an initial test suite, and $|F_{\min}|$ is the number of faults exposed by test cases in a reduced test suite subset. If the difference between $|F|$ and $|F_{\min}|$ is not significant, then FDE Loss is small. The lower the evaluated FDE Loss in the test suite subset, the better the fault detection effectiveness maintained by the algorithm.

- (i) The *t-test*. In many applications, the quantity of interest can be represented by the expectation of the model distribution. If one wants to know whether this expectation deviates from some a priori specified value, this can be investigated by

means of the *t*-test.⁵¹ Thus, in order to investigate whether or not the improvements of the fault detection effectiveness, utilizing our proposed methods, were significant, we adopted the *t*-test for matched pairs in this paper.^{51,18} The *t*-test is a parametric test assuming normality and equal variance of the two data samples.⁴⁵ The *t*-test helps developers to compare whether two groups have similar or different average values. Matched-pair is one type of experiment in which the samples are independent: That is, there is no relationship between the observations in one sample and the second sample. However, each observation in one sample is matched with an observation in the other sample.⁵² In this paper, the test starts with a null hypothesis assuming no significant improvement of the weighted combinatorial algorithms. Next, we evaluate the *p*-value by setting the number of distinct faults found by some selected traditional algorithms and in our approaches as a matched-pair. Moreover, a confidence level of 95% means that the assumption of the null hypothesis is rejected. That is, if the *p*-value is lower than 0.05, our approaches are able to enhance the fault detection effectiveness statistically significantly.

Finally, it should be noted that if we take advantage of these two criteria simultaneously, the FTT ratio is considered. Thus, we will also use the FTT ratio to evaluate whether the quality of the selected test cases is improved by our approaches or not. The larger the FTT ratio, the better the capability of test case to detect faults.

5.3 | Experimental results and analysis

We now analyze the overall performance of our proposed weighted combinatorial algorithms compared with some selected traditional algorithms.

5.3.1 | Reduced test suite size

First, Table 2 demonstrates the average size of the initial test suite of the object programs and the average size of the reduced test suite of the object programs of original HGS, original GRE, original GE, original greedy, and our proposed weighted combinatorial algorithms. The average SSR values of the four traditional test suite reduction algorithms and our proposed weighted combinatorial algorithms are provided in Table 3. Notice each number in a pair of brackets represents the rank of SSR value of each weighted combinatorial algorithm. The larger the SSR, the better the capability of the suite size reduction rate.

We can also be observed in Table 3, 56.875% of the SSR values were superior to traditional algorithms' values, and 20.625% of the values were the same as one of the traditional algorithms. In other words, 77.5% of the data support our proposed approaches as performing the same as or better than traditional algorithms. Except for the WCH, most of the SSR of the WC-algorithms surpassed the traditional algorithms. Although there were still some SSR values lower than the traditional values, the differences were minor. Among the five weighted methods, we conclude WA and WCU perform the best in suite size reduction.

5.3.2 | Fault detection effectiveness loss

Table 4 depicts the average number of faults that can be exposed by the selected test cases with the HGS, GRE, GE, greedy, and our proposed weighted combinatorial algorithms. In this table, $|F|$ means the number of faults exposed by test cases in an initial test suite, whereas $|F_{min}|$ denotes the number of faults exposed by test cases in a reduced test suite subset. The average values of *FDE Loss* of the four traditional algorithms and all weighted combinatorial algorithms are also provided in Table 5. Each number in a pair of brackets represents the rank of *FDE Loss* value of each weighted combinatorial algorithm. Notice that the smaller the *FDE Loss*, the better the fault detection effectiveness. For *totinfo* and *schedule*, the WCH- and WCU-algorithms perform with almost the lowest *FDE Loss*; however, for *tcas*, the WH-algorithm provides the best fault detection effectiveness. For *schedule2*, the WCU-algorithm has the best performance in *FDE Loss*. As for *printtokens* and *printtokens2*, the WH-, WCH-, and WCU-algorithms perform similarly. For *replace* and *space*, there is no weighted combinatorial algorithm that outperforms any others, whereas all the WC-algorithms perform better in fault detection effectiveness than the traditional algorithms.

TABLE 2 Number of chosen test cases by traditional algorithms and WC-algorithms

Programs	Algorithm	T	$ T_{min} $					
			Original	WA	WG	WH	WCH	WCU
totinfo	HGS	16.4	5.3	5	5.3	5.1	5.2	5.8
	GRE	29.4	5	5	5	5	5.1	5
	GE	29.4	5	5	5	5	5.3	5
	Greedy	16.7	5.3	5	5	5.3	5.1	5.3
tcas	HGS	55.6	5	5	5	5.1	5	5
	GRE	15.2	5	5	5	5	5	5
	GE	16.8	5	5	5	5.3	5	5
	Greedy	9.1	5	5	5	5	5	5
schedule	HGS	14.8	3.1	3	3.1	3.2	3.5	3.4
	GRE	77.7	3	3	3.2	3.2	3.4	3
	GE	49.9	3.3	3.7	3.3	3.3	3	3.3
	Greedy	16.2	3.4	3.5	3.4	3.4	3.3	2.6
schedule2	HGS	18.8	4.9	4.5	4.9	5.1	5	5.2
	GRE	127.8	4.6	4.8	5.2	4.8	5	4.5
	GE	36.7	4.7	4	4.7	5	5.3	4.7
	Greedy	16.7	4.9	5	5.6	4.9	5	5.3
printtokens	HGS	30	6.9	6.4	6.9	7	7.3	9.2
	GRE	23.3	6.6	6.5	6.4	6.6	6.4	6
	GE	90.2	8	8	5.7	7	7.3	7
	Greedy	15.5	7.6	7.3	7.2	7.6	6.8	5.7
printtokens2	HGS	41.3	6.2	5.4	6.2	6.4	7.1	9.2
	GRE	26.5	5.4	5.8	5.9	6.2	5.4	5.5
	GE	96	5.7	5.7	5.7	4.7	5.7	6
	Greedy	21.0	6.6	6.3	6.5	6.6	7.2	5.3
replace	HGS	81.5	10.7	10.5	10.7	11.2	11.5	12.2
	GRE	63.6	11	11.1	10.6	11	11	11.5
	GE	51.3	11	13.3	11	11	11.3	11
	Greedy	22.1	11.9	11.9	12	12.5	11.8	11
space	HGS	4452.2	117	108	117	113	107	119
	GRE	3460	114	113	114	112	118	110
	GE	1016	119	115	109	114	112	110
	Greedy	346.6	119	115.5	117	119	116	115

On the other hand, from the overall data in Table 5, 71.875% of the *FDE Loss* we see values are lower than traditional algorithms' values, and 11.875% of the values are the same as those of traditional algorithms. In other words, 83.75% of the data demonstrate our proposed approaches match or outperform traditional algorithms' fault detection. Most of the weighted combinatorial algorithms perform well in these eight object programs. From these results, we can state our proposed weighted combinatorial algorithms expose more faults than traditional algorithms because we adopted the evaluation metrics with secondary criteria to break ties.

5.3.3 | A *t*-test for matched pairs

Here, we assumed no significant improvement of the weighted combinatorial algorithms, and set it as H_0 , which is a null hypothesis. We performed a *t*-test for matched pairs and evaluated the *p*-values to examine if there is fault detection effectiveness improvement from utilizing weighted combinatorial algorithms. *p*-Values lower than 0.05 represent a statistically

TABLE 3 SSR values of traditional algorithms and WC-algorithms

Programs	Algorithm	SSR					
		Original	WA	WG	WH	WCH	WCU
totinfo	HGS	0.67	0.94	0.67	0.81	0.40	0.98
	GRE	0.89	0.94	0.88	0.60	0.54	0.97
	GE	0.83	0.83	0.82	0.83	0.83	0.89
	Greedy	0.55	0.70	0.70	0.55	0.69	0.54
tcas	HGS	0.85	0.85	0.91	0.40	0.86	0.75
	GRE	0.78	0.83	0.77	0.92	0.92	0.90
	GE	0.69	0.69	0.69	0.71	0.72	0.82
	Greedy	0.32	0.45	0.46	0.32	0.53	0.50
schedule	HGS	0.79	0.96	0.79	0.85	0.78	0.88
	GRE	0.93	0.96	0.94	0.69	0.68	0.93
	GE	0.91	0.96	0.91	0.91	0.96	0.94
	Greedy	0.69	0.78	0.58	0.69	0.56	0.76
schedule2	HGS	0.74	0.90	0.74	0.86	0.83	0.89
	GRE	0.86	0.93	0.86	0.67	0.91	0.93
	GE	0.80	0.93	0.80	0.90	0.80	0.95
	Greedy	0.60	0.70	0.43	0.60	0.52	0.68
printtokens	HGS	0.76	0.94	0.77	0.76	0.60	0.88
	GRE	0.70	0.86	0.91	0.70	0.76	0.93
	GE	0.85	0.85	0.95	0.78	0.93	0.91
	Greedy	0.57	0.53	0.60	0.57	0.62	0.64
printtokens2	HGS	0.85	0.95	0.85	0.76	0.66	0.97
	GRE	0.86	0.90	0.89	0.68	0.86	0.93
	GE	0.93	0.95	0.93	0.95	0.91	0.94
	Greedy	0.42	0.70	0.73	0.42	0.54	0.72
replace	HGS	0.82	0.82	0.87	0.80	0.63	0.73
	GRE	0.83	0.70	0.84	0.73	0.83	0.75
	GE	0.73	0.70	0.78	0.80	0.85	0.78
	Greedy	0.47	0.47	0.57	0.49	0.44	0.44
space	HGS	0.90	0.96	0.95	0.98	0.99	0.90
	GRE	0.89	0.97	0.89	0.92	0.65	0.93
	GE	0.88	0.95	0.96	0.96	0.89	0.95
	Greedy	0.57	0.60	0.50	0.57	0.64	0.59

significant difference between two groups of data at a 95% confidence level. Similarly, if p -values are lower than 0.01, this denotes the two groups of data are differ at a 99% confidence level. We utilize this statistical method to prove our proposed approaches are better than traditional algorithms.⁵¹ Thus, in this study, we reject the H_0 if a $p < 0.05$. Here, we take the average fault numbers from traditional algorithms and from weighted combinatorial algorithms as the first group of data and the second group of data, respectively. From Tables 6–9, at least 10% of the p -values in each WC-algorithm is less than 0.05. Note that a symbol “ns” is used to indicate that $p > 0.05$, meaning the difference is not statistically significant. For *space*, the WC-GRE and WC-GE demonstrate a significant difference ($p < 0.01$) between the traditional and our proposed algorithms. Furthermore, the WC-HGS and WC-Greedy exhibited a significant difference ($0.05 \geq p \geq 0.01$) between the traditional and our proposed algorithms. The experimental results reveal that up to 93.75% of our proposed approaches have significant differences with the traditional algorithms, supporting the effectiveness our methods to enhance fault detection.

TABLE 4 Number of faults exposed by traditional algorithms and WC-algorithms

Programs	Algorithm	F	$ F_{min} $					
			Original	WA	WG	WH	WCH	WCU
totinfo	HGS	19.8	12.7	13.3	11.5	11.9	12.4	15.4
	GRE	19.9	13	15.6	13	13.3	13.7	18
	GE	13.8	9	11.3	15.3	10.3	12.7	13
	Greedy	15.3	12.0	12.7	11.8	11.6	12.2	10.7
tcas	HGS	18.1	6.8	7.2	7.1	7.6	7	9.4
	GRE	13	6.1	7.3	6.2	7.5	6.8	9
	GE	15.8	8.3	7.3	8.3	9	9.3	7.7
	Greedy	8.7	6.3	6.4	6.4	6.8	7.3	4
schedule	HGS	5.8	2.1	2.2	2.2	2.3	2	4.2
	GRE	5.7	2.2	2.2	2.2	2	2.1	3.5
	GE	4.8	1.3	3.3	1.3	1.3	3	3.3
	Greedy	3.9	1.8	2.3	1.9	2.1	2.3	2.7
schedule2	HGS	5	2.4	2.5	2.3	2.2	2.1	3.8
	GRE	5.3	1.9	2.1	1.9	2.1	1.9	3
	GE	3.5	1.7	2.7	1.7	2	1.3	4.7
	Greedy	2.4	1.8	1.8	1.5	2.0	1.7	1.7
printtokens	HGS	5	3.1	3.1	2.8	2.8	2.8	3.6
	GRE	4.4	2.8	2.8	2.8	2.8	2.8	3.5
	GE	4.4	2	2	4	2.3	3	3.7
	Greedy	3.1	2.9	2.8	2.6	2.9	2.8	3
printtokens2	HGS	9.4	7.5	7.6	7.1	7.2	7.5	7.4
	GRE	8.5	7.6	7.6	7.4	7.6	7.6	7.5
	GE	9.6	6.3	6.7	7.3	7	7.7	8.3
	Greedy	8.3	7.4	7.7	7.9	7.1	7.5	7.3
replace	HGS	17.7	7.9	9.3	9.3	9.2	8.4	9.8
	GRE	16.6	9	8.2	8.4	8.7	9	11
	GE	21.1	7	10.7	10.7	8.7	10.3	10.7
	Greedy	11.3	8.4	9.2	8.0	7.6	8.7	9
space	HGS	32.9	26	27	25	28	28	26
	GRE	35	27	28	30	31	29	27
	GE	33	25	30	31	30	28	27
	Greedy	29.5	26.5	27	26.5	26.5	27.5	29

5.3.4 | Fault-to-test ratio

Table 10 depicts the fault-to-test (FTT) ratio values of the four traditional algorithms and all our proposed weighted combinatorial algorithms. Each number in a pair of brackets represents the rank of FTT ratio value of each of the weighted combinatorial algorithms. Notice the larger the FTT ratio, the more effective the chosen test case. The WCU- and WA-algorithms are the top two, exhibiting the best FTT ratio in *totinfo*, *schedule*, *schedule2*, and *replace*. In *tcas*, the WH-algorithm performs the steadiest, whereas the WCH-GE, WCH-Greedy, WCU-HGS, and WCU-GRE perform the best. As in *printtokens*, *printtokens2*, and *space*, our proposed weighted combinatorial algorithms have about the same FTT ratio, whereas they are still better than traditional algorithms. As Table 10 shows, 68.125% of the experimental results are larger than traditional values, and 14.375% of the experimental results are the same as traditional values. In other words, 82.5% of the data support our proposed approaches perform the same as or better than traditional algorithms. Thus, we conclude our proposed weighted combinatorial algorithms enhance the ability of exposing faults while keeping the same suite size reduction capability or vice versa.

TABLE 5 FDE loss values of traditional algorithms and WC-algorithms

Programs	Algorithm	FDE Loss					
		Original	WA	WG	WH	WCH	WCU
totinfo	HGS	0.26	0.33	0.17	0.23	0.11	0.24
	GRE	0.25	0.23	0.25	0.13	0.13	0.11
	GE	0.41	0.28	0.29	0.33	0.15	0.26
	Greedy	0.19	0.14	0.18	0.13	0.17	0.11
tcas	HGS	0.56	0.61	0.69	0.21	0.65	0.30
	GRE	0.67	0.55	0.53	0.67	0.72	0.55
	GE	0.45	0.34	0.45	0.34	0.43	0.37
	Greedy	0.28	0.27	0.33	0.17	0.24	0.20
schedule	HGS	0.52	0.62	0.52	0.52	0.52	0.22
	GRE	0.65	0.62	0.51	0.47	0.34	0.30
	GE	0.68	0.54	0.68	0.64	0.54	0.23
	Greedy	0.53	0.41	0.28	0.29	0.23	0.17
schedule2	HGS	0.41	0.51	0.27	0.34	0.33	0.26
	GRE	0.41	0.55	0.41	0.39	0.50	0.32
	GE	0.61	0.47	0.61	0.52	0.25	0.10
	Greedy	0.25	0.25	0.22	0.12	0.14	0.11
printtokens	HGS	0.28	0.38	0.23	0.23	0.15	0.16
	GRE	0.28	0.35	0.3	0.22	0.17	0.11
	GE	0.42	0.42	0.17	0.19	0.44	0.17
	Greedy	0.08	0.10	0.16	0.08	0.09	0.18
printtokens2	HGS	0.16	0.19	0.16	0.11	0.05	0.14
	GRE	0.17	0.13	0.17	0.08	0.17	0.11
	GE	0.27	0.28	0.24	0.30	0.11	0.07
	Greedy	0.09	0.09	0.05	0.08	0.03	0.18
replace	HGS	0.51	0.50	0.51	0.46	0.34	0.50
	GRE	0.53	0.44	0.53	0.43	0.50	0.23
	GE	0.40	0.26	0.31	0.31	0.38	0.31
	Greedy	0.26	0.19	0.25	0.21	0.18	0.33
space	HGS	0.21	0.23	0.11	0.20	0.20	0.21
	GRE	0.18	0.18	0.14	0.11	0.09	0.23
	GE	0.24	0.12	0.09	0.14	0.15	0.21
	Greedy	0.77	0.13	0.55	0.71	0.82	0.06

TABLE 6 Evaluated *p*-values of t-tests for WC-HGS

Programs	WA	WG	WH	WCH	WCU
totinfo	0.0001	0.0131	ns	0.0010	0.0483
tcas	0.0095	ns	0.0007	0.0010	0.0483
schedule	0.0124	0.0069	0.0047	0.0000	0.0483
schedule2	0.0012	0.0001	0.0007	0.0124	0.0483
printtokens	0.0007	ns	0.0124	0.0124	0.0483
printtokens2	0.0012	0.0001	0.0012	0.0003	0.0013
replace	0.0000	0.0000	0.0000	0.0000	0.0000
space	0.0000	0.0281	0.0220	0.0352	0.0352

Here “ns” denotes “not significant”.

TABLE 7 Evaluated *p*-values of *t*-tests for WC-GRE

Programs	WA	WG	WH	WCH	WCU
totinfo	0.0000	0.0003	0.0000	0.0002	0.0368
tcas	0.0000	0.0000	0.0008	0.0000	0.0033
schedule	ns	0.0416	ns	0.0124	0.0125
schedule2	0.0124	0.0124	ns	0.0124	0.0426
printtokens	0.0124	0.0012	0.0124	ns	0.0315
printtokens2	0.0003	0.0007	0.0124	0.0001	0.0315
replace	0.0000	0.0001	0.0018	0.0000	0.0376
space	0.0013	0.0004	0.0005	0.0013	0.0151

Here “ns” denotes “not significant”.

TABLE 8 Evaluated *p*-values of *t*-tests for WC-GE

Programs	WA	WG	WH	WCH	WCU
totinfo	0.0362	0.0332	0.0354	0.0000	0.0102
tcas	0.0027	0.0291	0.0002	0.0132	0.0001
schedule	0.0297	0.0000	0.0310	0.0299	0.0003
schedule2	0.0151	0.0000	0.0304	0.0000	0.0102
printtokens	0.0032	0.0403	0.0331	0.0039	0.0102
printtokens2	0.0102	0.0102	0.0102	0.0329	0.0102
replace	0.0117	0.0000	0.0000	0.0315	0.0001
space	0.0043	0.0016	0.0052	0.0026	0.0378

5.4 | Threats to validity

In this section, we address threats to this study’s validity. Adequate validity indicates the experimental results have a high ability to solve similar problems. Here, we would like to briefly address the current study’s threats to internal, external, and construct validity.⁵³ Threats to internal validity primarily involve the uncontrolled factors that affect experimental results. Some factors may influence internal validity. For instance, two of the most important threats to internal validity include the collection process of subject programs and the accuracy of subject programs. These selected subject programs have long been popular as experimental artifacts because they have been publicly released (or announced) for a long period of time. All the programs, faulty versions, and test pools used in our experiments were available from SIR.²¹

Additionally, in this paper, we simultaneously considered the branch coverage and du-pair coverage as the criteria.^{11,18} First, the branch coverage is a measure of the percentage of the Boolean expressions that the program has been invoked to both a true and false state in the testing. Notice that it is frequently common in the stage of unit testing to keep track of the branch coverage of the developed program. Complete coverage is not usually obtained. Second, the du-pair is a pairing of definition and use of a variable with at least one definition-clear path between them. Actually, the motivation for using

TABLE 9 Evaluated *p*-values of *t*-tests for WC-greedy

Programs	WA	WG	WH	WCH	WCU
totinfo	0.0000	0.0339	0.0124	0.0000	0.0497
tcas	0.0007	0.0000	0.0000	0.0000	0.0378
schedule	0.0000	0.0339	0.0000	0.0012	0.0378
schedule2	ns	ns	ns	0.0124	0.0378
printtokens	0.0007	0.0124	0.0124	ns	0.0378
printtokens2	ns	0.0000	0.0012	0.0124	0.0338
replace	0.0000	0.0000	0.0000	0.0000	0.0125
space	0.0282	0.0448	0.0181	0.0408	0.0352

Here “ns” denotes “not significant”.

TABLE 10 *FTT* ratio values of traditional algorithms and WC-algorithms

Programs	Algorithm	<i>FTT</i> ratio					
		Original	WA	WG	WH	WCH	WCU
totinfo	HGS	2.18	2.65	2.18	2.33	2.38	3.08
	GRE	0.78	3.12	2.61	0.65	2.68	3.60
	GE	1.8	2.27	3.07	2.07	2.38	2.60
	Greedy	2.16	2.54	2.35	2.18	2.39	2.00
tcas	HGS	1.42	1.44	1.42	1.49	1.39	1.88
	GRE	1.23	1.45	1.24	1.49	1.36	1.80
	GE	1.67	1.47	1.67	1.69	1.87	1.53
	Greedy	1.27	1.28	1.27	1.36	1.47	0.80
schedule	HGS	0.72	0.74	0.72	0.71	0.57	1.31
	GRE	0.72	0.72	0.70	0.63	0.62	1.02
	GE	0.4	0.91	0.40	0.40	1.00	1.00
	Greedy	0.54	0.66	0.55	0.61	0.70	1.03
schedule2	HGS	0.46	0.55	0.46	0.42	0.42	0.76
	GRE	0.37	0.43	0.37	0.43	0.39	0.66
	GE	0.36	0.67	0.36	0.40	0.25	1.00
	Greedy	0.34	0.36	0.27	0.40	0.34	0.31
printtokens	HGS	0.41	0.49	0.41	0.40	0.39	0.53
	GRE	0.36	0.43	0.43	0.43	0.43	0.51
	GE	0.25	0.25	0.71	0.33	0.41	0.52
	Greedy	0.39	0.38	0.37	0.39	0.41	0.53
printtokens2	HGS	1.15	1.4	1.15	1.12	1.06	1.23
	GRE	1.41	1.31	1.25	1.23	1.41	1.42
	GE	1.24	1.18	1.29	1.50	1.35	1.39
	Greedy	1.04	1.17	1.21	1.07	1.04	1.44
replace	HGS	0.87	0.89	0.87	0.82	0.73	0.58
	GRE	0.79	0.74	0.79	0.79	0.82	0.95
	GE	0.71	0.80	0.97	0.79	0.65	0.97
	Greedy	0.56	0.77	0.67	0.61	0.74	0.82
space	HGS	0.21	0.25	0.21	0.25	0.26	0.22
	GRE	0.25	0.25	0.26	0.28	0.25	0.26
	GE	0.23	0.26	0.25	0.25	0.25	0.25
	Greedy	0.21	0.22	0.23	0.22	0.24	0.25

two kinds of code-based testing criteria lies in the fact that when there is more than one test case of equal importance to a suite, the elimination of the redundant test cases encounters a tied condition, and the secondary criterion will help to break such ties during the minimizing process. Another reason that we select two kinds of criteria is that the branch and the du-pair coverages are, respectively, both control flow-based and data flow-based testing strategies.³⁹ Essentially, the data flow criteria are typically concerned with definition-clear paths from definition to use of individual variables.⁵⁴ Furthermore, the control flow of the program is represented in a flow graph, but in data flow-based testing, the control flow graph is annotated with information about how the program variables are defined and used. By using these two kinds of criteria, with our proposed approaches, we may identify a set of test cases that can exercise different structural and functional elements and improve the fault detection capability of the test suites reduction.

To attain sufficient test suites for each program, we first randomly choose varying numbers of test cases from the associated test pool, add them to the suite, and analyze the branch coverage based on the chosen test cases. If the chosen test cases do not cover all requirements, we added some additional test cases to achieve 100% branch coverage. To allow different levels of redundancy, we initially choose a random number of test cases to add to each suite varied over sizes

ranging from 0 to 0.5 times the number of lines of code in the program. We generated in total 200 test suites for each program. Each program was hand-instrumented to record the branch coverage information. For instance, we assigned a branch flag to every branch. When we executed the program, the flag is set to 1 if the test case has run through the branch. Similarly, to determine where the variable is defined and used, we need to assign a du-pair flag to check the data flow coverage. During this process, we defined a detailed rule to keep the human operation as objective as possible to guarantee good quality for all selected subject programs in this paper. We also carefully reviewed all the codes we produced for the experiments before starting. Another threat to internal validity was the implementation of traditional test suite reduction techniques. To diminish this threat, we implemented all traditional techniques very carefully, following their original algorithmic details previously published.^{8,26}

Threats to external validity are concerned with whether the same result of a given study is generalizable to other settings or situations. To minimize threats to external validity, for our experiments, we have used eight subject programs from a diverse range of Siemens suite programs: *tcas*, *schedule*, *schedule2*, *totinfo*, *printtokens*, *printtokens2*, *replace*, and *space*. To evaluate the efficiency of our proposed methods in this paper, our experiments were performed based on eight selected subject programs, ranging from 162 LOCs to 9127 LOCs. It should be noted that for *space*, the correct version as well as the 38 faulty versions and a test suite of 13,585 test cases can be downloaded from SIR. *Space* is 9126 LOCs, which is a somewhat larger program among the eight selected subject programs. The Siemens and *space* programs have for several years seen widespread use by many researchers. Thus, to make a fair performance comparison between our proposed methods and some traditional test suite reduction algorithms, it would be more convincing if experiments were performed based on commonly used programs. Therefore, the threats to external validity should be minimal.

Finally, construct validity typically refers to whether or not a measure exactly reflects or assesses the underlying construct that it is intended to measure. Therefore, the construct validity of our study could be affected by the number of adopted criteria in the experiment. In this study, four criteria (i.e., as SSR, FDE Loss, *t*-test, and FTT Ratio) are used to compare the performance of the traditional techniques with our proposed test suite reduction techniques—this could be useful for relieving threats to construct validity. It should be noted that most of our chosen criteria have been extensively adopted for evaluation in many test suite reduction studies.^{18,30} In general, a method or model that performs well according to one criterion could perform poorly according to another. However, we acknowledge that most criteria have limitations and strengths. Consequently, to check the performance of our proposed approaches, make a fairly comprehensive comparison with other existing algorithms, and avoid bias, we use four criteria in our paper.

6 | THREE-OBJECTIVES BINARY ILP MODEL

As mentioned in the preceding section, test suite size reduction tends to find the minimal test suite subset that can satisfy all the requirements of the original test suite. It is worth noting that a minimal set cover problem can also be expressed as a binary ILP problem by encoding test-case criteria as objective functions of a binary ILP problem. Hsu and Orso⁹ claimed computed solutions are optimal for the minimization problem. This binary ILP representation is allowed for the application of mathematical programming techniques to compute minimal cover sizes.³⁴ A binary ILP model differs from traditional HGS, GRE, GE, or greedy algorithms—having a subjective function and an objective function. Onsjö and Danielsson⁵⁵ reported the ILP model runtime was vastly lower than that of NSGA-II. This value does not lie solely in the reduced runtime but also in the increased capabilities for the solver in solving a larger multiobjective problem. Additionally, Zhang et al.⁴¹ also demonstrated that ILP always achieves the largest values of test-suite size for their selected subject programs compared to the greedy, HGS, and GRE techniques. They³⁴ also proposed utilizing ILP for test-case selection, and traditional techniques of test-case prioritization for prioritizing selected test cases. Concerning fault detection, their experimental results indicated the ILP-based approach outperforms their selected approaches (such as GAs, random prioritization, total statement-coverage prioritization, additional statement-coverage prioritization, and total method-coverage prioritization) for general and version-specific prioritizations. Hou et al.⁵³ proposed a method of quota-constrained test-case prioritization for regression testing of service-centric systems (SCSs). They divided the testing time into time slots and iteratively selected and prioritized test cases for each time slot using the ILP model. Their experimental results demonstrated their proposed methods could schedule test cases for execution with higher effectiveness in achieving total branch coverage and exposing faults with the constraint of request quotas. Two studies^{34,53} have demonstrated cost-constraint problems can also be analyzed using the ILP model.

We adopt the evaluation metrics that we applied in the preceding section as the objective function in a binary ILP model. In addition to using control flow coverage and data flow coverage as our testing criteria, we further consider the capability

of fault detection for each test case to enhance the number of exposed faults. Although the characteristics of the binary ILP model proposed in Black et al. differ from the weighted combinatorial algorithms we proposed, the concept of using bi-criteria is similar. Thus, we attempt to combine control flow and data flow coverage with the binary ILP model and modify the objective function to see whether this model can perform as well as our proposed approaches.

6.1 | Evaluation metric

To improve this approach, we combine the $WCEI(t)$, which we used in the WC-HGS, WC-GRE, WC-GE, and WC-Greedy algorithms, with the three-objective binary ILP model. Here we plan to use branch coverage, du-pair coverage, and the capability of fault detection as our testing criteria. We intend to use the binary ILP model to find test cases that cover all branches, du-pairs, and faults. Again, there may be some branches, du-pairs, or faults in some object programs that cannot be covered by any test case. This makes it impossible to find feasible sets to achieve 100% coverage. To avoid this situation, we check the test cases and set the requirements that can never be covered as covered. Before using the ILP model, we first evaluate the $WCEI(t)$. In contrast to the evaluation metrics mentioned in Section 4.2, we have three terms. The following is the formula of $WCEI(t)$:

$$WCEI(t) = \frac{1}{3} \times WCEI_{br}(t) + \frac{1}{3} \times WCEI_{du}(t) + \frac{1}{3} \times WCEI_f(t) \quad (16)$$

Again, $WCEI_{br}(t)$ and $WCEI_{du}(t)$ represent the weighted combinatorial EIreplaceability of branch and du-pair coverage for a test case t , respectively. Here, we introduce one more criterion, $WCEI_f(t)$, which denotes the weighted combinatorial EIreplaceability of fault coverage for test case t . We make these three coverages equally important by assigning the same weighting factors. We next sum them up and obtain $WCEI(t)$ as our total contribution of each selected test case.

6.2 | Three-objectives binary ILP model

Combining the evaluation metric above, the final formulation of our model is as follows:

$$\text{Minimize} \left(\alpha \frac{\sum_{j=1}^{|Q|} x_j}{\sum_{j=1}^{|Q|} 1} + (1 - \alpha) \frac{\sum_{j=1}^{|Q|} \frac{x_j}{WCEI(t_j)}}{\sum_{j=1}^{|Q|} \frac{1}{WCEI(t_j)}} \right) \quad (17)$$

$$\text{Subject to} \begin{cases} \sum_{j=1}^{|Q|} br_{ij}x_j \geq 1, i = 1, \dots, |S_{br}| \\ \sum_{j=1}^{|Q|} du_{ij}x_j \geq 1, i = 1, \dots, |S_{du}| \\ \sum_{j=1}^{|Q|} ft_{ij}x_j \geq 1, i = 1, \dots, |S_f| \end{cases} \quad (18)$$

First, we identify the set of requirements (including branch, du-pairs, and fault coverage) covered by each test case. Next, we group these as Q . $|Q|$ is the number of sets in Q . α is the weighting factor that we use to assign the weight to our minimization of the number of test sets and the maximization of the $WCEI(t)$. S_{br} , S_{du} , and S_f represent the set of all branch requirements, du-pair requirements, and faults, respectively. There are three subjective functions in this model, each representing one requirement in the subject program. In these subject functions, br_{ij} , du_{ij} and ft_{ij} are the constraint co-efficient matrix, which denote branch, du-pair, and fault requirements, respectively. These three subjective functions ensure each requirement (including branches, du-pairs, and faults) is covered by at least one of the test cases selected in the subset. In Equations (17) and (18), x_j is the binary decision variable that represents whether we choose it or not. It is an array of Boolean expressions that the program has been invoked to for both a true and false in testing.

FIGURE 9 The pseudo code of three-objectives ILP model

```

1 algorithm Three-objectives ILP model
2
3 input
4  $TS$ : original generated (unreduced) test suite
5  $C$ : set of all control flow requirements
6  $D$ : set of all data flow requirements
7  $F$ : set of all fault requirements
8  $S^c = \{S_1^c, S_2^c, \dots, S_n^c\}$ :  $S_i^c$  contains test cases which satisfy control flow requirement  $c_i$ 
9  $S^d = \{S_1^d, S_2^d, \dots, S_m^d\}$ :  $S_i^d$  contains test cases which satisfy data flow requirement  $d_i$ 
10  $S^f = \{S_1^f, S_2^f, \dots, S_p^f\}$ :  $S_i^f$  contains test cases which satisfy fault requirement  $f_i$ 
11 output
12  $RS$ : a representative reduced set of  $TS$ 
13
14 begin
15  $RS \leftarrow \{\}$ ;
16 for each test case  $t$  in generated test suite  $TS$  do
17    $WCEI[t] \leftarrow EvaluateWCEI(t, TS, S^c, S^d, S^f)$ ;
18 end for
19 // ILP Model
20  $\alpha \leftarrow 0.5$ ;
21  $Req\_C \leftarrow \{\}$ ;
22  $Req\_D \leftarrow \{\}$ ;
23  $Req\_F \leftarrow \{\}$ ;
24 for-each test case  $t$  in generated test suite  $TS$  do
25    $Req\_C \leftarrow Req\_C \cup Req_t^C$ ; //  $Req_t^C$  is the set of requirements of control flow covered by  $t$ 
26    $Req\_D \leftarrow Req\_D \cup Req_t^D$ ; //  $Req_t^D$  is the set of requirements of data flow covered by  $t$ 
27    $Req\_F \leftarrow Req\_F \cup Req_t^F$ ; //  $Req_t^F$  is the set of fault covered by  $t$ 
28 end-for
29  $Q \leftarrow \{Req\_C, Req\_D, Req\_F\}$ ;
30 for  $j = 1$  to  $|Q|$  do
31    $Object1 \leftarrow Object1 + x_j$ ;
32    $Object2 \leftarrow Object2 + x_j / WCEI[t_j]$ ;
33    $temp \leftarrow temp + 1 / WCEI[t_j]$ ;
34 end-for
35  $Object1 \leftarrow Object1 / |Q|$ ;
36  $Object2 \leftarrow Object2 / temp$ ;
37  $f \leftarrow \alpha * Object1 + (1 - \alpha) * Object2$ ; //  $f$  is the objective function
38  $RS \leftarrow ILP(f, TS, Q, WCEI[t], C, D, F)$ ;
39 return  $RS$ ;
40 end

```

In our approach, the objective function is combined with the number of test cases and the reciprocal of $WCEI(t)$. The first objective function intends to minimize the number of test cases needed to cover all branches, du-pairs, and faults. The second objective function seeks to maximize the total score of $WCEI(t)$. In this manner, we prevent any one of the objective functions from dominating the result. We then set the weighting factor α as 0.5, which makes selecting minimal test cases to cover all requirements as important as the total score of $WCEI(t)$, for all selected test cases. We can adjust the weighting factor to whether we want to make the number of test cases or the score of $WCEI(t)$ more important. The pseudo code of our proposed algorithm is provided in Figure 9.

6.3 | Experimental results of three-objectives binary ILP model

The average SSR values of the proposed three-objective ILP and the four traditional algorithms (HGS, GRE, GE, and greedy) are provided in Table 11. We intend to achieve 100% branch coverage, du-pair coverage, and fault coverage. Thus, we can see the reduced suite size of our three-objective ILP is more than all of the other four algorithms. However, its SSR still performs better than the four traditional algorithms. From Table 12, it is apparent that the fault exposed by our proposed three-objective ILP is more than any other algorithm. Furthermore, its FDE Loss for all programs is 0. This result demonstrates our proposed three-objective ILP significantly enhances fault detection effectiveness. Our proposed three-objective ILP selects more test cases than the four traditional algorithms to ensure the branch coverage, du-pair coverage, and fault coverage reach 100%. However, from Table 13, we see our proposed approach performs a better FTT ratio in all object programs, except for *printtokens2*.

In our proposed approach, although we did not tune the weighting factors as we did in previous weighted combinatorial algorithms. Instead, we normalized them to prevent any one of the fractions from dominating the whole model. In

TABLE 11 SSR values of traditional algorithms and three-objectives binary ILP model

Programs		ILP	HGS	GRE	GE	Greedy
totinfo	$ T $	229.2	16.4	29.4	29.4	16.7
	$ T_{min} $	6.8	5.3	5	5	5.3
	SSR	0.97	0.68	0.83	0.83	0.55
tcas	$ T $	957.6	55.6	15.2	16.8	9.1
	$ T_{min} $	16.2	5	5	5	5
	SSR	0.98	0.91	0.67	0.70	0.32
schedule	$ T $	417.4	14.8	77.7	49.9	16.2
	$ T_{min} $	8.2	3.1	3	3.3	3.4
	SSR	0.98	0.79	0.96	0.93	0.69
schedule2	$ T $	736.2	18.8	127.8	36.7	16.7
	$ T_{min} $	7.4	4.9	4.6	4.7	4.9
	SSR	0.99	0.74	0.96	0.87	0.60
printtokens	$ T $	462.4	30	23.3	90.2	15.5
	$ T_{min} $	8.2	6.9	6.6	8	7.6
	SSR	0.98	0.77	0.72	0.91	0.57
printtokens2	$ T $	283.2	41.3	26.5	96	21
	$ T_{min} $	9	6.2	5.4	5.7	6.6
	SSR	0.97	0.85	0.80	0.94	0.42
replace	$ T $	2733.8	81.5	63.6	51.3	22.1
	$ T_{min} $	25.6	10.7	11	11	11.9
	SSR	0.99	0.87	0.83	0.79	0.47
space	$ T $	4329.2	4452.2	3460	1016	346.6
	$ T_{min} $	132	117	114	119	119
	SSR	0.97	0.97	0.97	0.88	0.57

conclusion, our proposed three-objective ILP model not only enhances the performance of suite size reduction but also significantly improves fault detection effectiveness.

7 | CONCLUSION

During software testing, test teams usually have a well-specified set of test cases and attempt to find potential faults within time and resource limitations to enhance software quality. A test suite is a collection of test cases that are grouped and run together. One of the most common methods includes using test suite reduction to eliminate redundant cases or to select representative test cases. However, in traditional test suite reduction techniques, random selection is typically applied when a tie occurs. This may lead to excluding test cases that have higher fault detection capability. In this paper, we have proposed some weighted combinatorial algorithms to enhance the tie-breaking techniques. We have first considered both the control flow and the data flow as our testing criteria and combine them by using a weighted means with four existing test suite reduction algorithms. A GA is used to find the best combination of weighting factors to assign to the two testing criteria. We conducted the experiments on real programs on SIR. Experimental results demonstrate the proposed weighted combinatorial algorithms perform better on test suite reduction and also on fault detection capability. For example, the SSR of GE and WA-GE for program *schedule2* are 0.8 and 0.93, respectively; the fault detection capability of WA-GE is enhanced by 14%. For the program *space*, the SSR of HGS and WG-HGS are 0.9 and 0.95, respectively; the fault detection capability of WG-HGS is increased by 10%. In addition, a three-objective binary ILP model is also proposed and discussed. The experiments show our proposed methods enhance the quality of the reduced test suite subset, which means a more effective test suite reduction process for practitioners.

Additionally, there remain some open issues. First, there could exist a better and more rigorous tool profiling the sub-program than manual flag insertion. Second, the metaheuristic techniques, such as GA, have been implemented in a

TABLE 12 *FDE* loss values of traditional algorithms and three-objectives binary ILP model

Programs		ILP	HGS	GRE	GE	Greedy
totinfo	$ F $	23	19.8	19.9	13.8	15.3
	$ F_{min} $	23	12.7	13	9	12
	<i>FDE Loss</i>	0	0.36	0.35	0.35	0.22
tcas	$ F $	41	18.1	13	15.8	8.7
	$ F_{min} $	41	6.8	6.1	8.3	6.3
	<i>FDE Loss</i>	0	0.62	0.53	0.47	0.28
schedule	$ F $	9	5.8	5.7	4.8	3.9
	$ F_{min} $	9	2.1	2.2	1.3	1.8
	<i>FDE Loss</i>	0	0.64	0.61	0.73	0.50
schedule2	$ F $	10	5	5.3	3.5	2.4
	$ F_{min} $	10	2.4	1.9	1.7	1.8
	<i>FDE Loss</i>	0	0.52	0.64	0.51	0.25
printtokens	$ F $	7	5	4.4	4.4	3.1
	$ F_{min} $	7	3.1	2.8	2	2.9
	<i>FDE Loss</i>	0	0.38	0.36	0.55	0.06
printtokens2	$ F $	10	9.4	8.5	9.6	8.3
	$ F_{min} $	10	7.5	7.6	6.3	7.4
	<i>FDE Loss</i>	0	0.20	0.11	0.34	0.11
replace	$ F $	32	17.7	16.6	21.1	11.3
	$ F_{min} $	32	7.9	9	7	8.4
	<i>FDE Loss</i>	0	0.55	0.46	0.67	0.26
space	$ F $	38	32.9	35	33	29.5
	$ F_{min} $	38	26	27	25	26.5
	<i>FDE Loss</i>	0	0.21	0.23	0.24	0.10

TABLE 13 *FTT* ratio values of traditional algorithms and three-objective binary ILP model

Programs	ILP	HGS	GRE	GE	Greedy
totinfo	3.4	2.18	0.78	1.8	2.16
tcas	2.54	1.42	1.23	1.67	1.27
schedule	1.15	0.72	0.72	0.4	0.54
schedule2	1.36	0.46	0.37	0.36	0.34
printtokens	0.87	0.41	0.36	0.25	0.39
printtokens2	1.15	1.15	1.41	1.24	1.04
replace	1.26	0.87	0.79	0.71	0.56
space	0.29	0.21	0.25	0.23	0.21

wide range of applications, whereas previous studies have also indicated the software engineering area is suitable to apply these techniques. It is still possible to use improved or enhanced GAs or other algorithms to find better weighting factor assignment methods to reducing the time it takes to find weighting factors. In this paper, we only use branch coverage and du-pair coverage in our empirical study to test if the control flow and the data flow requirement can be combined properly. In fact, two or more different kinds of testing criteria, such as the modified condition/decision coverage and the all-paths coverage, can be used to perform a further and more detailed analysis. Third, besides SIR programs, more subject programs collected from a diverse range of projects can be used for performance comparisons of different algorithms. Fourth, we can discuss the changes and/or improvements of software reliability when our proposed methods and traditional test suite reduction algorithms are used.

Finally, in order to investigate whether the improvements of the fault detection effectiveness utilizing our proposed methods were significant, we adopted the parametric *t*-test for matched pairs. However, it should be noted that Yoo and

Harman³⁵ selected the one-sided Wilcoxon signed-rank test. The Wilcoxon signed rank test is a non-parametric test and is used when the data in two samples are paired. According to Yoo and Harman's studies,³⁵ the null hypothesis is that with two different algorithms, the mean values of the size of Pareto-frontiers are equivalent to the number of solutions that are not dominated by the reference Pareto frontier. In general, the decision on which statistical test (i.e., parametric vs. non-parametric) to select for analysis can be aided by using a decision tree. We plan to explore the above-mentioned issues and publish related results in the near future.

ACKNOWLEDGMENTS

The work described in this paper was supported by the Ministry of Science and Technology, Taiwan, under Grant MOST 108-2221-E-007-033-MY3. The authors wish to thank anonymous referees for their constructive and insightful suggestions, which led to a significant improvement of this paper.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in Software-artifact Infrastructure Repository at <http://sir.unl.edu/portal/>, reference number.⁴⁹

ORCID

Chin-Yu Huang  <https://orcid.org/0000-0003-4931-4572>

REFERENCES

1. Ammann P, Offutt J. *Introduction to Software Testing*. Cambridge University Press; 2008.
2. Musa JD, Iannino A, Okumoto K. *Software Reliability: Measurement, Prediction, Application*. NY, USA: McGraw-Hill; 1987.
3. Jorgensen PC. *Software Testing: A Craftsman's Approach*. 4th ed. CRC Press; 2013.
4. Binkley D. Semantics guided regression test cost reduction. *IEEE Trans Software Eng*. 1997;23(8):498-516.
5. Kalaei A, Rafe V. An optimal solution for test case generation using ROBDD graph and PSO algorithm. *Qual Reliab Eng Int*. 2016;32(7):2263-2279.
6. Naik K, Tripathy P. *Software Testing and Quality Assurance: Theory and Practice*. John Wiley & Sons Inc; 2008.
7. Yoo S, Harman M. Regression testing minimisation, selection and prioritization: a survey, software testing. *Verification and Reliability*. 2012;22(2):67-120.
8. Harrold MJ, Gupta R, Soffa ML. A methodology for controlling the size of a test suite. *ACM Trans Softw Eng Methodol*. 1993;2(3):270-285.
9. Hsu HY, Orso A. MINTS: a general framework and tool for supporting test suite minimization. Proceedings of the 31st IEEE International Conference on Software Engineering. Vancouver, BC; 2009;419-429.
10. Jeffrey D, Gupta N. Test suite reduction with selective redundancy. Proceedings of the 21st IEEE International Conference on Software Maintenance (ICSM). 2005;549-558.
11. Lin JW, Huang CY. Analysis of test suite reduction with enhanced tie-breaking techniques. *Info Softw Technol*. 2009;51(4):679-690.
12. Huang CY, Chen CS, Lai CE. Evaluation and analysis of incorporating fuzzy expert system approach into test suite reduction. *Info Softw Technol*. 2016;79:79-105.
13. Chiang CL. Analysis of experimental studies on weighted combinatorial testing criterion for software test suite reduction. Master's thesis, Hsinchu, Taiwan: Institute of Information Systems and Applications, National Tsing Hua University; 2016.
14. Sampath S, Bryce RC. Improving the effectiveness of test suite reduction for user-session-based testing of web applications. *Info Softw Technol*. 2012;54(7):724-738.
15. Lin CT, Tang KW, Kapfhammer GM. Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests. *Inf Softw Technol*. 2014;56(10):1322-1344.
16. Zhu H. A formal analysis of the subsume relation between software test adequacy criteria. *IEEE Trans Softw Eng*. 1996;22(4):248-255.
17. Kandil P, Moussa S, Badr N. Cluster-based test cases prioritization and selection technique for agile regression testing. *J Softw: Evol Process*. 2017;29:e1794.
18. Jeffrey D, Gupta N. Improving fault detection capability by selectively retaining test cases during test suite reduction. *IEEE Trans Softw Eng*. 2007;33(2):108-123.
19. Malaiya YK, Li NX, Bieman J, Karcich R, Skibbe B. The relationship between test coverage and reliability. Proceedings of 1994 IEEE International Symposium on Software Reliability Engineering. Monterey, CA, USA; 1994:186-195. <https://doi.org/10.1109/ISSRE.1994.341373>
20. Santelices R, Jones JA, Yu YB, Harrold MJ. Lightweight fault-localization using multiple coverage types. 2009 IEEE 31st International Conference on Software Engineering. Vancouver, BC; 2009:56-66. <https://doi.org/10.1109/ICSE.2009.507050>
21. Hutchins M, Foster H, Goradia T, Ostrand T. Experiments on the effectiveness of dataflow- and controlflow-based test adequacy criteria. Proceedings of the 16th International Conference on Software Engineering (ICSE '94). CA, USA; 1994:191-200.
22. Chen CS. A study of test suite reduction with the fuzzy expert system approach. Master's thesis, Department of Computer Science, National Tsing Hua University; 2015.

23. Chiu CY. Analysis and comparison of weighted combinatorial algorithms for test suite reduction. Master's thesis, Hsinchu, Taiwan: Department of Computer Science, National Tsing Hua University; 2015.
24. Chen TY, Lau MF. Heuristics towards the optimization of the size of a test suite. *WIT Transactions on Information and Communications Technologies*. 1995;14:415-424.
25. Chen TY, Lau MF. A simulation study on some heuristics for test suite reduction. *Info Softw Technol*. 1998;40(13):777-787.
26. Chvatal V. A greedy heuristic for the set-covering problem. *Math Oper Res*. 1979;4(3):233-235.
27. Vokolos FI, Frankl PG. Empirical evaluation of the textual differencing regression testing technique. Proceedings of the 14th International Conference on Software Maintenance. Bethesda, MD; 1998:44-53.
28. Heimdahl MPE, George D. Test-suite reduction for model based tests: effects on test quality and implications for testing. Proceedings of the 19th International Conference on Automated Software Engineering. 2004:176-185.
29. Mansour N, El-Fakih K. Simulated annealing and genetic algorithms for optimal regression testing. *J Soft Maint: Research and Practice*. 1999;11(1):19-34.
30. Rothermel G, Harrold MJ, von Ronne J, Hong C. Empirical studies of test-suite reduction, software testing. *Verification and Reliability*. 2002;12(4):219-249.
31. Shi A, Gyori A, Gligoric M, Zaytsev A, Marinov D. Balancing trade-offs in test-suite reduction. Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering. Hong Kong, China; 2014:246-256.
32. Di Nardo D, Alshahwan N, Briand L, Briand L. Coverage- based regression test case selection, minimization and prioritization: a case study on an industrial system, software testing. *Verification and Reliability*. 2015;25(4):371-396.
33. Rothermel G, Untch RH, Chu C, Harrold MJ. Prioritizing Test Cases for Regression Testing. *IEEE Trans Softw Eng*. 2001;27(10):929-948.
34. Zhang L, Hou SS, Guo C, Xie T, Mei H. Time-aware test-case prioritization using integer linear programming. Proceedings of the 18th International Symposium on Software Testing and Analysis (ISSTA '09). Chicago, IL; 2009:213-224.
35. Yoo S, Harman M. Using hybrid algorithm for pareto Efficient multi-objective test suite minimization. *J Syst Softw*. 2010;83(4):689-701.
36. Eghbali S, Kudva V, Rothermel G, Tahvildari L. Supervised tie breaking in test case prioritization. Proceedings of the 41st International Conference on Software Engineering: Companion Proceedings. Montreal, Quebec, Canada; May 2019:242-243.
37. Coviello C, Romano S, Scanniello G, Marchetto A, Antoniol G, Corazza A. Clustering support for inadequate test suite reduction. Proceedings of 2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER): Campobasso. Italy, Mar. 2018:95-105.
38. Cruciani E, Miranda B, Verdecchia R, Bertolino A. Scalable approaches for test suite reduction. Proceedings of the 41st International Conference on Software Engineering. Montreal, Quebec, Canada; May 2019:419-429.
39. Howden WE. Reliability of the path analysis testing strategy. *IEEE Trans Softw Eng*. 1976;SE-2(3):208-215.
40. Frankl PG, Weiss SN, Weyuker EJ. ASSET: a system to select and evaluate tests. Proceedings of IEEE Conference on Software Tools. New York; 1985:72-79.
41. Zhang L, Marinov D, Zhang L, Khurshid S. An empirical study of JUnit test-suite reduction. Proceedings of the 22nd International Symposium on Software Reliability Engineering. Hiroshima; 2011:170-179.
42. Bullen PS. *Handbook of Means and their Inequalities: v 2.0*. Kluwer Academic Publishers; 2003.
43. Marichal JL. Aggregation operators for multicriteria decision aid, doctoral dissertation: Department of management. Belgium: University of Liege; 1998.
44. Jones JA, Harrold MJ. Test-suite reduction and prioritization for Modified condition/decision coverage. *IEEE Trans Softw Eng*. 2003;29(3):195-209.
45. Haupt RL, Haupt SE. *Practical Genetic Algorithms*. 2nd ed. Hoboken: John Wiley & Sons Inc. Publication; 2004.
46. Abdul-Rahman O, Munetomo M, Akama K. An adaptive resolution hybrid binary-real coded genetic algorithm. *Artificial Life and Robotics*. 2011;16(1):121-124.
47. Lu KD, Zeng GQ, Chen J, et al. Comparison of binary coded genetic algorithms with different selection strategies for continuous optimization problems: 2013 Chinese Automation Congress, Changsha; 2013:364-368.
48. Clarke J, Dolado JJ, Harman M, et al. Reformulating software engineering as a search problem. *IEE Proceedings-Software*. 2003;150:161-175.
49. Software-artifact Infrastructure Repository, 2015, available at <http://sir.unl.edu/portal/>
50. Rothermel G, Harrold MJ, Ostrin J, Hong C. An empirical study of the effects of minimization on the fault detection capabilities of test suites. Proceedings of the International Conference on Software Maintenance. Bethesda, MD; 1998:34-43.
51. Kpodjedo S, Ricca F, Antoniol G, Galinier P. Evolution and search based metrics to improve defects prediction. Proceedings of the 1st International Symposium on Search Based Software Engineering: Windsor; 2009:23-32.
52. Keller G. Managerial Statistics: v 9.0, 2012.
53. Hou SS, Zhang L, Xie T, Sun JS. Quota-constrained test-case prioritization for regression testing of service-centric systems, Proceedings of 24th IEEE International Conference on Software Maintenance (ICSM 2008). Beijing, China; Sept-Oct. 2008:257-266.
54. Pezze M, Young M. *Software Testing and Analysis: Process, Principles and Techniques*. New York, NY, USA: John Wiley & Sons, Inc; 2007.
55. Onsjö K, Danielsson A. *Retrofitting of Housing Blocks Minimizing Space Heating and Cost using Combinatorial Optimization Modelling and Algorithms*. Göteborg: Chalmers University of Technology, Gothenburg, Sweden; 2018.

AUTHOR BIOGRAPHIES

Chi-Lun Chiang received the BE degree (2014) in Department of Computer Science from National Tsing Hua University, Hsinchu, Taiwan, and the ME degree (2016) in the Institute of Information Systems and Applications from National Tsing Hua University, Hsinchu, Taiwan. Her research interests include software testing and software metrics.

Chin-Yu Huang is a Full Professor in the Department of Computer Science at National Tsing Hua University (NTHU), Hsinchu, Taiwan. He received the MS (1994), and the PhD (2000) in Electrical Engineering from National Taiwan University, Taipei, Taiwan. He was with the Bank of Taiwan from 1994 to 1999, and was a senior software engineer at Taiwan Semiconductor Manufacturing Company from 1999 to 2000. Before joining NTHU in 2003, he was a division chief of the Central Bank of China, Taipei. He received the Ta-You Wu Memorial Award from the National Science Council of Taiwan in 2008. He also received an Honorable Mention Best Paper Award in the 2010 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM 2010). In 2010, he was ranked at the 7th place of top scholars in Systems and Software Engineering worldwide between 2004 and 2008 by the Journal of Systems and Software based on his research on software reliability, software testing, and software metrics. His research interests are software reliability engineering, software testing, software metrics, and fault tree analysis. He has published over 100 papers in these areas.

Chang-Yu Chiu received the BE degree (2013) in Department of Computer Science from National Tsing Hua University, Hsinchu, Taiwan and the ME degree (2015) in Department of Computer Science from National Tsing Hua University, Hsinchu, Taiwan. He is currently a software engineer at TSMC (Taiwan Semiconductor Manufacturing Company), Hsinchu, Taiwan. His research interests include software testing and software measurement.

Kai-Wen Chen received the BE degree (2018) in Department of Computer Science from National Tsing Hua University, Hsinchu, Taiwan, and the ME degree (2020) in Department of Computer Science from National Tsing Hua University, Hsinchu, Taiwan. He is currently a software engineer in the Cyberlink Corporation, Taiwan. His research interests include software maintenance, software categorization and software testing.

Chen-Hua Lee received the BE degree (2017) in the Department of Information Management from National Central University, Taoyuan, Taiwan. She was a software testing engineer at LINE Taiwan from 2018 to 2019. She received the ME degree (2021) in the Institute of Information Systems and Applications from National Tsing Hua University, Hsinchu, Taiwan. Her research focuses on software testing and software reliability.

SUPPORTING INFORMATION

Additional supporting information may be found in the online version of the article at the publisher's website.

How to cite this article: Chiang C-L, Huang C-Y, Chiu C-Y, Chen K-W, Lee C-H. Analysis and assessment of weighted combinatorial criterion for test suite reduction. *Qual Reliab Eng Int*. 2022;38:358–388.

<https://doi.org/10.1002/qre.2984>